IOT-BASED SMART PRE-PAID ENERGY METER WITH PRIORITIZED LOAD SWITCHING

ABSTRACT

With the increasing demand for efficient energy management, real-time monitoring of electricity consumption has become essential. This project presents an IoT-based energy meter monitoring system that enables continuous tracking and analysis of energy usage. By integrating IoT technology with a microcontroller, the system collects and transmits consumption data to a centralized IoT server, allowing users and utility providers to remotely monitor electricity usage. The system also facilitates remote load control, enabling users to turn the load on or off via the server. A threshold-based alert mechanism notifies users of excessive consumption, and if the prepaid energy limit is exceeded, an additional charge is applied for each extra unit consumed. This approach enhances billing accuracy, optimizes energy consumption, and improves revenue collection for utility companies. The proposed solution contributes to smart energy management by leveraging IoT for real-time monitoring, promoting sustainable consumption, and enhancing overall grid efficiency.

CHAPTER 1

INTRODUCTION

Energy consumption is increasing rapidly due to urbanization and technological advancements, making efficient electricity management essential. Traditional energy meters require manual readings, leading to inaccuracies, billing delays, and a lack of real-time monitoring. To overcome these challenges, an IoT-based energy meter monitoring system is proposed, enabling real-time tracking of electricity usage and improving billing accuracy.

This system integrates an IoT modem, relay, LCD, buzzer, and energy meter with a microcontroller programmed in Embedded C. The energy meter continuously records power consumption and transmits data to an IoT server, providing real-time access to the electricity department while displaying the readings on an LCD for user awareness. A buzzer is activated when energy usage reaches a set threshold, and if the consumption continues beyond the limit, the buzzer rings for every additional unit consumed, adding a penalty until the user resets the system.

By offering real-time monitoring, automated alerts, and penalty-based tracking, this system encourages energy efficiency, helps users manage their consumption, and enhances billing transparency. It benefits both consumers and utility providers by reducing billing disputes and optimizing electricity distribution.

OVERVIEW:

The increasing demand for electricity due to urbanization and technological advancements has made efficient energy management a necessity. Traditional energy meters rely on manual readings, leading to inefficiencies, inaccuracies, and delays in billing. Consumers often receive their bills at the end of the month without any real-time awareness of their consumption, resulting in unexpected high costs. Additionally, manual meter readings are prone to human errors, leading to disputes between consumers and electricity providers. To address these challenges, an IoT-based energy meter monitoring system is proposed, which enables real-time tracking of electricity consumption and automated alerts to improve energy efficiency and billing accuracy.

The proposed system integrates an IoT modem, relay, LCD, buzzer, and energy meter with a microcontroller programmed in Embedded C. The energy meter continuously records electricity usage, and the IoT modem transmits this data to an IoT server, allowing electricity

authorities to monitor usage remotely. The LCD display provides real-time updates, ensuring that consumers are always aware of their electricity consumption. This system eliminates the need for manual readings and provides automated data updates, ensuring accuracy and reducing human intervention.

A key feature of the system is the buzzer alert mechanism, which helps users regulate their energy usage. When consumption reaches a predefined threshold, the buzzer activates to notify the user. If the user continues consuming electricity beyond this limit, the buzzer rings continuously for every additional unit consumed, and a penalty amount is added to the bill. This continues until the user manually resets the system. This feature discourages excessive electricity consumption and encourages responsible usage.

The IoT-based energy meter monitoring system provides several advantages over conventional meters. Real-time monitoring enables consumers to track their energy usage at any moment, allowing them to take immediate action to reduce unnecessary consumption. Automated alerts help users stay within their desired energy limits, preventing bill shocks at the end of the month. Additionally, this system enhances billing accuracy by eliminating human errors associated with manual meter readings. Since data is transmitted directly to the electricity department, there is no need for personnel to visit each household, making the billing process more efficient.

Another significant advantage of this system is its ability to optimize energy distribution. With real-time data collection, utility providers can analyze electricity consumption patterns and make better decisions regarding load distribution. This ensures that power is allocated efficiently, reducing the chances of overloads and power outages. Moreover, the penalty mechanism serves as a deterrent for excessive electricity use, promoting energy conservation and cost savings for both consumers and electricity providers.

Unlike traditional energy meters, which only provide consumption details at the end of the billing cycle, this IoT-based system ensures continuous access to energy usage data. This transparency allows users to monitor their electricity usage and adjust their habits accordingly. By making consumers more conscious of their power consumption, this system contributes to overall energy efficiency and promotes sustainable energy usage practices.

The IoT-based energy meter monitoring system represents a significant advancement in smart energy management. By integrating real-time tracking, automated alerts, and penalty-based monitoring, the system enhances energy efficiency, improves billing transparency, and provides a smarter approach to electricity usage. It benefits both consumers and utility

providers by offering a more automated, efficient, and cost-effective solution for managing electricity consumption. As the need for smarter energy solutions grows, this system serves as a step toward modernizing power monitoring and improving overall energy management.

MOTIVATION:

Electricity consumption is rising, making efficient energy management essential. Traditional meters require manual readings, leading to inaccuracies, billing delays, and unexpected high bills. Consumers lack real-time monitoring, making it difficult to regulate their usage. Manual readings also introduce human errors, causing billing disputes.

This project aims to develop an IoT-based energy meter monitoring system for real-time tracking and accurate billing. The system integrates an IoT modem, energy meter, LCD, and buzzer with a microcontroller. When consumption reaches a set threshold, the buzzer alerts the user. If usage continues, the buzzer rings for every extra unit consumed, adding a penalty until reset.

By automating meter readings, the system ensures billing transparency and reduces human errors. Real-time monitoring helps users control their energy consumption and avoid excessive charges. The system also aids electricity providers in optimizing power distribution. With IoT integration, this project promotes smart energy management and sustainable electricity usage.

PROBLEM STATEMENT:

Despite the advancements in digital energy meters, several issues still exist in traditional electricity monitoring and billing systems. These challenges need to be addressed for better efficiency, accuracy, and ease of use for both consumers and utility providers. The major problems identified are as follows:

1. **Manual Meter Reading** – Traditional energy meters require a large workforce for meter readings, increasing operational costs and inefficiencies.
2. **High Workforce Requirement** – A significant number of employees are needed for tasks such as meter reading, billing, and payment collection, making the process resource-intensive.
3. **Billing Errors** – Mistakes occur due to human errors in reading meters, miscalculations, or estimated billing, leading to consumer disputes and financial losses.

4. **Time-Consuming Bill Payments** – Consumers often need to visit payment centers and stand in long queues for hours, making the process inconvenient.
5. **Lack of Real-Time Monitoring** – Consumers do not have real-time access to their electricity usage, leading to unintentional excessive consumption.
6. **Careless Electricity Usage** – Without awareness of real-time costs, consumers tend to use electricity inefficiently, resulting in high electricity bills.
7. **Delayed Response to Overconsumption** – Consumers are unaware of exceeding their budget until they receive the bill at the end of the cycle, preventing timely corrective actions.

To overcome these issues, an IoT-based energy meter monitoring system is proposed, ensuring real-time consumption tracking, automated alerts, and penalty-based monitoring to encourage responsible electricity usage while eliminating manual errors and inefficiencies.

AIMS AND OBJECTIVES:

1. **To develop an IoT-enabled energy meter system** that continuously monitors and records electricity consumption.
2. **To provide real-time data transmission** of energy usage to both consumers and electricity providers for better monitoring.
3. **To integrate a buzzer alert mechanism** that notifies users when their electricity consumption reaches a set threshold.
4. **To implement a penalty system** where the buzzer rings continuously for every extra unit consumed beyond the limit, encouraging responsible usage.
5. **To eliminate human errors in billing** by automating meter readings and reducing reliance on manual processes.
6. **To enhance billing transparency** by ensuring accurate readings and avoiding estimation-based bills.
7. **To reduce operational costs for utility providers** by minimizing the workforce required for meter readings and bill collection.
8. **To promote energy conservation** by making users aware of their real-time consumption and encouraging efficient usage.
9. **To improve consumer convenience** by enabling remote monitoring and reducing the need for standing in queues for bill payments.
10. **To support the transition to smart energy management** by integrating IoT technology for better electricity monitoring and usage control.

CHAPTER 2
LITERATURE REVIEW

With the increasing demand for efficient energy management, real-time monitoring of electricity consumption has become essential. Several studies have proposed IoT-based smart energy meters to address inefficiencies in traditional metering systems, such as manual billing errors, lack of real-time data, and power theft. (Sandhya A. et al.,2022) introduce an ESP32-based system that uploads consumption data to a cloud platform, allowing users to track their energy usage remotely. Similarly, (Afreen et al.2023) propose an ATMEGA328-based system with Wi-Fi connectivity, enabling real-time monitoring and reducing human errors in billing. These solutions demonstrate how IoT enhances consumer awareness and home automation.

Another key research area focuses on prepaid billing, offering a pay-before-use approach to eliminate billing delays and revenue losses for utility providers. (Bhavsar et al.,2019) propose an "Adaptive Meter" that integrates power metering with a prepaid system, allowing users to recharge remotely via GSM. (Shravani et al.,2023) further refines this model by introducing mobile app-based tracking, enabling consumers to monitor deductions, voltage fluctuations, and overall energy consumption. These systems enhance billing accuracy, providing transparency and control to consumers while streamlining utility company operations.

Additionally, bidirectional communication and power theft prevention are crucial components of smart metering. (Ranganayaki et al.,2021) propose an IoT-based prepaid energy metering system that enables two-way communication between the consumer and the utility provider. This ensures remote monitoring, immediate action against anomalies, and improved power distribution. These studies highlight the importance of integrating smart metering with IoT to enhance grid efficiency and security.

Our proposed project builds upon these concepts by integrating real-time energy tracking, prepaid billing, load control, and intelligent energy management into a unified system. By leveraging IoT and microcontroller-based technology, our system collects and transmits consumption data to a centralized server, enabling remote monitoring and control. Users can turn the load on or off through the IoT interface, and a threshold-based alert mechanism notifies them of excessive consumption. If the prepaid energy limit is exceeded,

an additional charge is applied per extra unit, ensuring better revenue collection for utility providers. Unlike existing solutions, our project also incorporates predictive analytics and dynamic tariff adjustments to optimize energy distribution and enhance efficiency.

This comprehensive approach ensures enhanced billing accuracy, optimized consumption, and improved grid management. By leveraging IoT for real-time monitoring, our solution promotes sustainable energy consumption, reduces wastage, and contributes to the modernization of smart grids, making energy management more reliable, automated, and efficient.

BirendrakumarSahani.al[1] are made a practical model of IoT Based Smart Energy Meter. The proposed model is used to calculate the energy consumption of the household, and even make the energy unit reading to be handy. It reduces the wastage of energy and bring awareness among all.

Mayur Rawte.al[2] are developed a system to solve many problems such as over usage of electricity, large amount of manpower transparency of usage and wastage of money and resources etc.This technology allows verified customers to check status of electricity usage by using Device identification number and password in real time. This can be done from web application using Internet.

NazmatToyin.al[3] are designed the system to resort to a local server and database, upon resumption of internet connection, all information are synchronized with the web server. The billing is handled locally by the web server and has not been interfaced with any online payment platform agencies.

Mst. ShahnajParvin.al[4] are explained the framework and how it will be beneficial in detecting an unauthorized use of electricity. The relative advantages of the proposed system over conventional systems have also been outlined in the paper.

Azfar Tufail.al [5] provide some enhancement in the conventional Metering system by smart metering. The term Smart Meter is an advanced energy meter that measures consumption of electrical energy providing additional information compared to a conventional energy meter.

Omijeh.al[6], introduced a tamper detect feature for a GSM solution for prepaid energy meter, however this work didnt provide an interactive interface for real-monitoring, access control as well as a robust database.

Mahfuzet al (2020) proposed the power monitoring of the consumed energy is one of the main concepts. Once the power is monitored it is sent to the consumer through SMS and a

feedback control is taken from the SMS to the microcontroller for the relay control. Mishra, J. K et al (2018) presented old meter is made as new smart meter that has an Electronic Meter Automation Device that sends the output data to the webpage and smart app for the real-time monitor of the power consumed. And the feedback control is connected in between webpage and microcontroller for the control of Electronic Meter.Ali Zaidi et al (2008) [7], presented in his work the proposed system consists of the digital billing and power consumption on lcd display.Gautam A. Raiker et al presented the paper the load is monitored in the web database through the IOT. And the energy management in a demand side is clearly mentioned what are the necessary things needs to be followed. On switching of the on load or off load is made simple.

In 2010, using Multi-appliance power disaggregation technology implementers implemented the linear detection algorithm to determine which appliances are active in their power contributions. Problems are robust to errors in this database. [8] In 2011, using cloud computing technology found the solution for efficiency calculation of individual equipment. [9] In 2012, using three feedback system, monitored the energy in residential Real-Time. It is critical to the continuing engagement and use of the device to save energy. Residences to determine the feedback provided by real-time energy monitors results in lower residential consumption rates during the 30 days after installation. [10] In 2013, using GREEN technology is the smallest Zigbee-compatible node in existence. This technology will possible in every place sensing of a different data types, from energy metering to environmental monitoring. [11] 1n 2014, GSM technology implemented automatic power will be reading. [12] In 2016. Using wifi technology application can develop for Apple and BlackBerry 10 OS, thus providing multiple platform users support [13] In 2017, using IOT technology An IoT device was created for measuring the voltage, current, power and energy of a three-phase four-line power line in a laboratory building [14] Through a brief review of the published literature and previously done work, we can say that the researches have done a severe work on the plc power line communication and Internet of Things (IoT). It is concluded from the ken study of their work that in today`s world PLC & IoT based meter could improve the overall efficiency of the existing or present system and could help in examining the unnecessary losses of power in different areas.

CHAPTER 3

EXISTING SYSTEM

The existing electricity metering system predominantly relies on conventional digital or analog energy meters, which require manual meter readings to track electricity consumption. In this traditional setup, utility personnel physically visit consumer premises, note down meter readings, and generate bills based on monthly usage. This manual process is not only time-consuming but also prone to human errors, inaccurate readings, and billing disputes. Additionally, consumers remain unaware of their electricity usage until they receive their bill, often leading to unexpectedly high charges.

Although some advancements have been made in prepaid energy meters, where users must recharge their accounts in advance to use electricity, these systems still lack real-time tracking and alert mechanisms to notify users when they exceed a set usage threshold. Consumers may unknowingly use excessive electricity, only realizing the impact when their balance runs out or their supply is cut off. This leads to inconvenience and prevents proactive energy management.

In an effort to enhance metering efficiency, GSM-based smart meters have been introduced. These meters can automatically transmit readings to electricity providers, reducing the need for manual meter readings. While this improves billing accuracy, it does not provide consumers with instant alerts or help them regulate their consumption. Users remain unaware of excessive energy usage until they receive their bill or their prepaid balance depletes. The system does not actively encourage responsible consumption through real-time feedback.

Limitations of the Existing System

1. **Manual Meter Readings** – Traditional energy meters require utility workers to physically check and record readings, leading to human errors, inefficiencies, and high operational costs.

2. **Delayed Billing and Awareness** – Consumers are billed at the end of the month and are often unaware of their real-time electricity consumption, making it difficult to manage usage effectively.
3. **No Real-Time Alerts** – Most existing systems lack an automated notification system to warn users when they reach a certain threshold of electricity consumption. This prevents timely corrective actions and results in uncontrolled usage.
4. **Billing Errors and Disputes** – Manual readings are prone to mistakes, which can lead to incorrect billing amounts and disputes between consumers and service providers. Estimated billing methods further increase inaccuracies.
5. **Lack of Automated Penalty System** – Existing meters do not include a mechanism to discourage excessive usage through automated penalties or alerts. Consumers may continue using electricity without being notified of extra charges until they receive their bill.
6. **Limited Remote Monitoring Capabilities** – While some advanced meters support remote reading transmission, they do not provide consumers with continuous, real-time tracking of their energy consumption on an easily accessible platform.
7. **Inefficient Energy Usage** – Without proper monitoring and timely feedback, users tend to consume more electricity than necessary, leading to higher bills and inefficient energy utilization.
8. **Dependency on Manual Bill Payment** – In many cases, consumers must visit payment centres or rely on third-party systems to pay their electricity bills, which is inconvenient and time-consuming.

Challenges in the Existing System

- **High manpower requirements** – Utility providers need a large workforce for meter readings and bill distribution, increasing operational costs.
- **Lack of real-time user control** – Consumers cannot track their usage continuously, preventing them from adjusting their consumption habits effectively.
- **Delayed response to overconsumption** – Since there is no instant alert system, users remain unaware of high consumption until their bill arrives.
- **Absence of smart automation** – The current system does not integrate smart IoT features to enhance monitoring, control, and energy conservation

CHAPTER 4
PROPOSED SYSTEM

The proposed system improves traditional energy meters by incorporating IoT-based real-time monitoring, an automated buzzer alert mechanism, and a penalty-based billing system to promote responsible electricity usage. Unlike conventional meters that provide monthly readings, this system continuously tracks power consumption and provides instant notifications, allowing users to stay aware of their energy usage. The system includes a microcontroller, IoT module, energy meter, LCD display, and a buzzer, all working together to provide accurate monitoring and immediate alerts when predefined limits are exceeded.

The digital energy meter records power usage in real time and transmits data to the microcontroller, which updates both the LCD display and IoT server for remote monitoring. This eliminates the need for manual meter readings, reducing human errors and ensuring transparency in billing. The LCD display provides real-time consumption details, enabling users to monitor their energy usage and take necessary steps to control excessive power consumption.

When energy usage crosses a set threshold, the buzzer is activated as a warning. Unlike conventional alerts that turn off quickly, this system ensures the buzzer continues ringing for every additional unit consumed beyond the limit. Additionally, a penalty-based billing mechanism adds extra charges for exceeding the set consumption, encouraging users to regulate their power usage. The system also allows users to reset the buzzer manually after acknowledging the warning, but if consumption continues to rise, the buzzer will reactivate, reinforcing awareness.

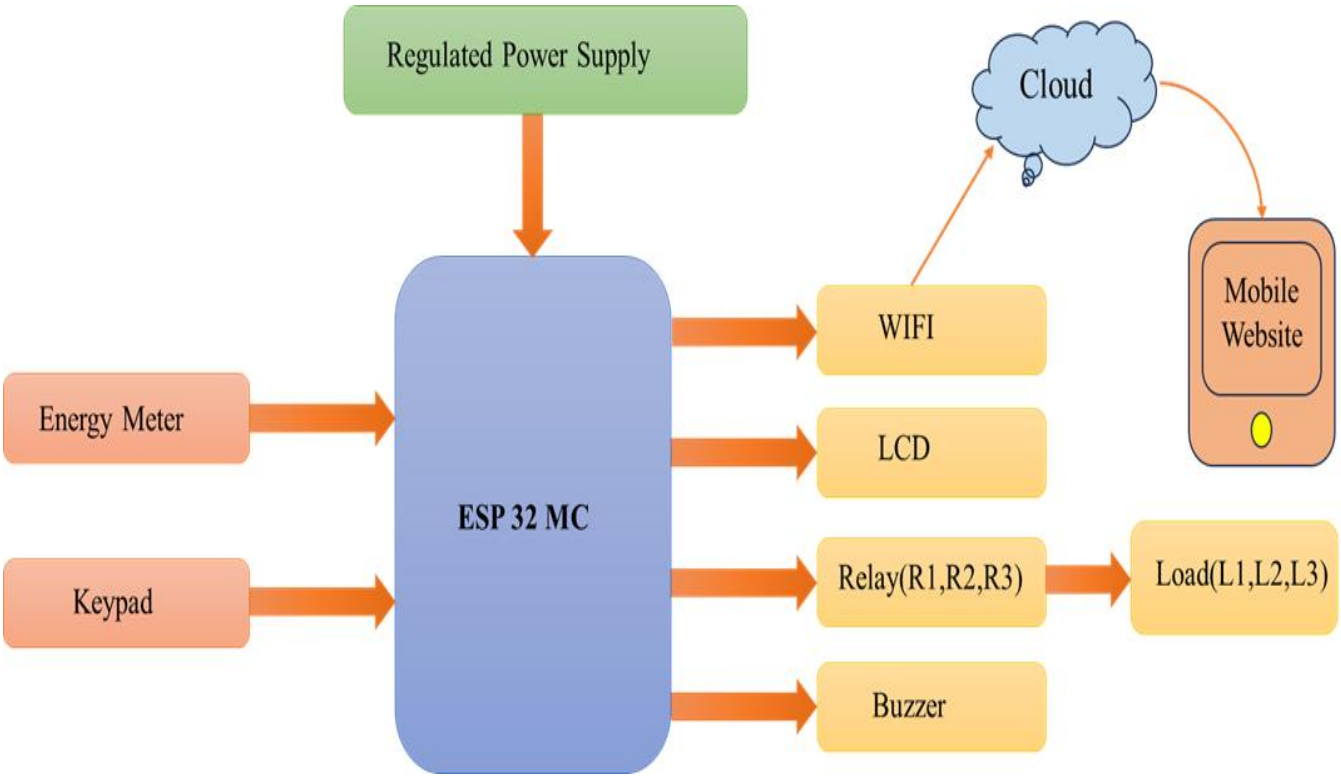
IoT connectivity enables real-time data transmission to an online server, eliminating the need for manual readings. Users can access their consumption records and analyze trends to optimize energy usage. The automated system reduces dependency on manual labor, eliminates billing errors, and enhances consumer awareness. Unlike prepaid meters that

disconnect power, this system does not cut electricity but instead encourages users to regulate consumption through continuous alerts and penalty-based charges.

By integrating real-time tracking, automated alerts, and a penalty-based billing system, this smart meter provides a modern, efficient, and cost-effective approach to energy management. It benefits consumers by reducing unexpected high bills and helping them make informed decisions about their electricity usage while also assisting power providers in efficient energy distribution. This system not only ensures better energy conservation but also contributes to a more sustainable and intelligent power management solution.

Unlike prepaid meters that disconnect power when the balance runs out, this system does not cut off electricity but instead encourages users to regulate their consumption through continuous alerts and penalty-based charges. This consumer-friendly approach ensures that users remain informed about their power usage while still enforcing responsible energy consumption practices. The proposed system introduces an efficient, automated, and user-friendly solution for energy monitoring, seamlessly integrating IoT technology to provide accurate, real-time data without manual intervention.

The adoption of this system will revolutionize energy metering by empowering consumers with real-time tracking tools, reducing electricity wastage, and promoting better energy conservation efforts. By providing accurate and timely consumption data, the system helps users make informed decisions, ultimately leading to lower energy costs and more sustainable electricity usage. This solution not only benefits individual consumers but also contributes to a more efficient and reliable power distribution network, making it a significant advancement in modern energy management systems.



BLOCK DIAGRAM

The block diagram represents a system built around the ESP32 microcontroller for energy monitoring and load control. A regulated power supply ensures stable operation of the ESP32 and connected components. The system receives input from an energy meter, which measures power consumption, and a keypad that allows user interaction.

The ESP32 processes data and communicates with different output devices. A WiFi module enables cloud integration, allowing real-time data access and control via a mobile website. The LCD provides a local display of system information. A relay module controls multiple loads, switching them on or off based on commands from the microcontroller. A buzzer is included to provide alerts or notifications.

Through cloud connectivity, users can monitor and control the system remotely using a mobile website, ensuring efficient energy management. The system integrates both local and remote control functionalities for better usability and monitoring.

ADVANTAGES:

- **Real-Time Monitoring** – Users can track their electricity consumption instantly through IoT-based real-time data transmission.
- **Automated Alerts** – The system provides continuous buzzer alerts when consumption exceeds the set limit, ensuring users remain aware of their usage.
- **Penalty-Based System** – Additional charges are applied for excess consumption, encouraging responsible energy usage and preventing wastage.
- **No Manual Meter Reading Required** – Eliminates human error and reduces the need for staff to take manual readings, ensuring accurate billing.
- **Remote Access** – Users and electricity providers can access consumption data remotely via an IoT server, improving efficiency and transparency.
- **Cost Savings** – Helps consumers reduce their electricity bills by providing timely alerts and encouraging controlled usage.
- **Data Logging & Analysis** – Stores historical usage data, allowing users and providers to analyse trends and optimize energy management.
- **No Power Disconnection** – Unlike prepaid meters, this system does not cut off electricity but instead warns users and imposes penalties for excess usage.

- **User-Friendly Interface** – The LCD display provides real-time updates, making it easy for users to monitor their energy consumption.
- **Suitable for Various Applications** – Can be implemented in residential, commercial, and industrial sectors for effective energy management.Real-Time Monitoring – Users can track their electricity consumption instantly through IoT-based real-time data transmission.

APPLICATIONS:

- **Residential Use** – Helps homeowners monitor and manage electricity consumption efficiently, reducing wastage and preventing unexpected high bills.
- **Commercial Establishments** – Enables businesses, offices, and retail stores to track energy usage, optimize consumption, and control operational costs.
- **Industrial Sector** – Assists factories and manufacturing units in monitoring large-scale power consumption and improving energy efficiency.
- **Educational Institutions** – Allows schools, colleges, and universities to regulate electricity usage and reduce unnecessary power consumption in classrooms and labs.
- **Hospitals & Healthcare Facilities** – Ensures uninterrupted power supply while monitoring excessive energy use to improve efficiency and cost-effectiveness.
- **Government & Public Buildings** – Helps government offices and public facilities track and optimize energy usage for better resource management.
- **Smart Cities & Urban Planning** – Supports smart energy management systems in urban areas, contributing to efficient power distribution and sustainability.
- **Rental Apartments & Gated Communities** – Provides tenants and property owners with real-time energy tracking, ensuring fair and transparent billing.
- **Hotels & Hospitality Industry** – Helps hotels monitor guest room energy usage, optimizing costs while maintaining comfort and efficiency.
- **Data Centres& IT Infrastructure** – Ensures proper monitoring of power consumption in data centres, preventing energy wastage and optimizing cooling systems.

CHAPTER 5

EMBEDDED SYSTEMS

5.1 Embedded Systems:

An embedded system is a computer system designed to perform one or a few dedicated functions often with real-time computing constraints. It is embedded as part of a complete device often including hardware and mechanical parts. By contrast, a general-purpose computer, such as a personal computer (PC), is designed to be flexible and to meet a wide range of end-user needs. Embedded systems control many devices in common use today.

Embedded systems are controlled by one or more main processing cores that are typically either microcontrollers or digital signal processors (DSP). The key characteristic, however, is being dedicated to handle a particular task, which may require very powerful processors. For example, air traffic control systems may usefully be viewed as embedded, even though they involve mainframe computers and dedicated regional and national networks between airports and radar sites. (Each radar probably includes one or more embedded systems of its own.)

Since the embedded system is dedicated to specific tasks, design engineers can optimize it to reduce the size and cost of the product and increase the reliability and performance. Some embedded systems are mass-produced, benefiting from economies of scale.

Physically embedded systems range from portable devices such as digital watches and MP3 players, to large stationary installations like traffic lights, factory

controllers, or the systems controlling nuclear power plants. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large chassis or enclosure.

In general, "embedded system" is not a strictly definable term, as most systems have some element of extensibility or programmability. For example, handheld computers share some elements with embedded systems such as the operating systems and microprocessors which power them, but they allow different applications to be loaded and peripherals to be connected. Moreover, even systems which don't expose programmability as a primary feature generally need to support software updates. On a continuum from "general purpose" to "embedded", large application systems will have subcomponents at most points even if the system as a whole is "designed to perform one or a few dedicated functions", and is thus appropriate to call "embedded". A modern example of embedded system is shown in fig: 2.1.

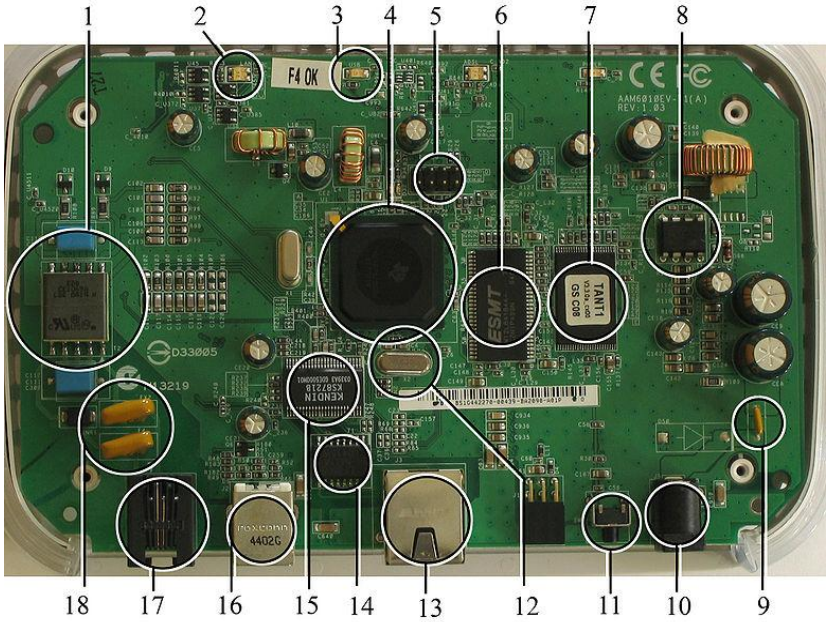


Fig 5.1:A modern example of embedded system

Labeled parts include microprocessor (4), RAM (6), flash memory (7).Embedded systems programming is not like normal PC programming. In many ways, programming for an embedded system is like programming PC 15 years ago. The hardware for the system is usually chosen to make the device as cheap as possible. Spending an extra dollar a unit in order to make things easier to program can cost millions. Hiring a programmer for an extra month is cheap in comparison. This means the programmer must make do with slow processors and low memory, while at the same time battling a need for

efficiency not seen in most PC applications. Below is a list of issues specific to the embedded field.

5.1.1 History:

In the earliest years of computers in the 1930–40s, computers were sometimes dedicated to a single task, but were far too large and expensive for most kinds of tasks performed by embedded computers of today. Over time however, the concept of programmable controllers evolved from traditional electromechanical sequencers, via solid state devices, to the use of computer technology.

One of the first recognizably modern embedded systems was the Apollo Guidance Computer, developed by Charles Stark Draper at the MIT Instrumentation Laboratory. At the project's inception, the Apollo guidance computer was considered the riskiest item in the Apollo project as it employed the then newly developed monolithic integrated circuits to reduce the size and weight. An early mass-produced embedded system was the Autonetics D-17 guidance computer for the Minuteman missile, released in 1961. It was built from transistor logic and had a hard disk for main memory. When the Minuteman II went into production in 1966, the D-17 was replaced with a new computer that was the first high-volume use of integrated circuits.

2.1.2 Tools:

Embedded development makes up a small fraction of total programming. There's also a large number of embedded architectures, unlike the PC world where 1 instruction set rules, and the Unix world where there's only 3 or 4 major ones. This means that the tools are more expensive. It also means that they're lowering featured, and less developed. On a major embedded project, at some point you will almost always find a compiler bug of some sort.

Debugging tools are another issue. Since you can't always run general programs on your embedded processor, you can't always run a debugger on it. This makes fixing your program difficult. Special hardware such as JTAG ports can overcome this issue in part. However, if you stop on a breakpoint when your system is controlling real world hardware (such as a motor), permanent equipment damage can occur. As a result, people doing embedded programming quickly become masters at using serial IO channels and error message style debugging.

5.1.3 Resources:

To save costs, embedded systems frequently have the cheapest processors that can do the job. This means your programs need to be written as efficiently as possible. When dealing with large data sets, issues like memory cache misses that never matter in PC programming can hurt you. Luckily, this won't happen too often- use reasonably efficient algorithms to start, and optimize only when necessary. Of course, normal profilers won't work well, due to the same reason debuggers don't work well.

Memory is also an issue. For the same cost savings reasons, embedded systems usually have the least memory they can get away with. That means their algorithms must be memory efficient (unlike in PC programs, you will frequently sacrifice processor time for memory, rather than the reverse). It also means you can't afford to leak memory. Embedded applications generally use deterministic memory techniques and avoid the default "new" and "malloc" functions, so that leaks can be found and eliminated more easily. Other resources programmers expect may not even exist. For example, most embedded processors do not have hardware FPUs (Floating-Point Processing Unit). These resources either need to be emulated in software, or avoided altogether.

5.1.4 Real Time Issues:

Embedded systems frequently control hardware, and must be able to respond to them in real time. Failure to do so could cause inaccuracy in measurements, or even damage hardware such as motors. This is made even more difficult by the lack of resources available. Almost all embedded systems need to be able to prioritize some tasks over others, and to be able to put off/skip low priority tasks such as UI in favor of high priority tasks like hardware control.

5.2 Need For Embedded Systems:

The uses of embedded systems are virtually limitless, because every day new products are introduced to the market that utilizes embedded computers in novel ways. In recent years, hardware such as microprocessors, microcontrollers, and FPGA chips have become much cheaper. So when implementing a new form of control, it's wiser to just buy the generic chip and write your own custom software for it. Producing a custom-made chip to handle a particular task or set of tasks costs far more time and money. Many embedded computers even come with extensive libraries, so that "writing your own software" becomes a very trivial task indeed. From an implementation viewpoint, there is a major difference between a computer and an embedded system. Embedded systems are often required to

provide Real-Time response. The main elements that make embedded systems unique are its reliability and ease in debugging.

5.2.1 Debugging:

Embedded debugging may be performed at different levels, depending on the facilities available. From simplest to most sophisticate they can be roughly grouped into the following areas:

- Interactive resident debugging, using the simple shell provided by the embedded operating system (e.g. Forth and Basic)
- External debugging using logging or serial port output to trace operation using either a monitor in flash or using a debug server like the Remedy Debugger which even works for heterogeneous multi core systems.
- An in-circuit debugger (ICD), a hardware device that connects to the microprocessor via a JTAG or Nexus interface. This allows the operation of the microprocessor to be controlled externally, but is typically restricted to specific debugging capabilities in the processor.
- An in-circuit emulator replaces the microprocessor with a simulated equivalent, providing full control over all aspects of the microprocessor.
- A complete emulator provides a simulation of all aspects of the hardware, allowing all of it to be controlled and modified and allowing debugging on a normal PC.
- Unless restricted to external debugging, the programmer can typically load and run software through the tools, view the code running in the processor, and start or stop its operation. The view of the code may be as assembly code or source-code.

Because an embedded system is often composed of a wide variety of elements, the debugging strategy may vary. For instance, debugging a software(and microprocessor) centric embedded system is different from debugging an embedded system where most of the processing is performed by peripherals (DSP, FPGA, co-processor). An increasing number of embedded systems today use more than one single processor core. A common problem with multi-core development is the proper synchronization of software execution. In such a case, the embedded system design may wish to check the data traffic on the busses between the processor cores, which requires very low-level debugging, at signal/bus level, with a logic analyzer, for instance.

5.2.2 Reliability:

Embedded systems often reside in machines that are expected to run continuously for years without errors and in some cases recover by them if an error occurs. Therefore the software is usually developed and tested more carefully than that for personal computers, and unreliable mechanical moving parts such as disk drives, switches or buttons are avoided.

Specific reliability issues may include:

- The system cannot safely be shut down for repair, or it is too inaccessible to repair. Examples include space systems, undersea cables, navigational beacons, bore-hole systems, and automobiles.
- The system must be kept running for safety reasons. "Limp modes" are less tolerable. Often backups are selected by an operator. Examples include aircraft navigation, reactor control systems, safety-critical chemical factory controls, train signals, engines on single-engine aircraft.
- The system will lose large amounts of money when shut down: Telephone switches, factory controls, bridge and elevator controls, funds transfer and market making, automated sales and service.

A variety of techniques are used, sometimes in combination, to recover from errors—both software bugs such as memory leaks, and also soft errors in the hardware:

- Watchdog timer that resets the computer unless the software periodically notifies the watchdog
- Subsystems with redundant spares that can be switched over to
- software "limp modes" that provide partial function
- Designing with a Trusted Computing Base (TCB) architecture[6] ensures a highly secure & reliable system environment
- An Embedded Hypervisor is able to provide secure encapsulation for any subsystem component, so that a compromised software component cannot interfere with other subsystems, or privileged-level system software. This encapsulation keeps faults from propagating from one subsystem to another, improving reliability. This may also allow a subsystem to be automatically shut down and restarted on fault detection.

- Immunity Aware Programming

5.3 Explanation of Embedded Systems:

5.3.1 Software Architecture:

There are several different types of software architecture in common use.

- Simple Control Loop:

In this design, the software simply has a loop. The loop calls subroutines, each of which manages a part of the hardware or software.

- Interrupt Controlled System:

Some embedded systems are predominantly interrupting controlled. This means that tasks performed by the system are triggered by different kinds of events. An interrupt could be generated for example by a timer in a predefined frequency, or by a serial port controller receiving a byte. These kinds of systems are used if event handlers need low latency and the event handlers are short and simple.

Usually these kinds of systems run a simple task in a main loop also, but this task is not very sensitive to unexpected delays. Sometimes the interrupt handler will add longer tasks to a queue structure. Later, after the interrupt handler has finished, these tasks are executed by the main loop. This method brings the system close to a multitasking kernel with discrete processes.

- Cooperative Multitasking:

A non-preemptive multitasking system is very similar to the simple control loop scheme, except that the loop is hidden in an API. The programmer defines a series of tasks, and each task gets its own environment to “run” in. When a task is idle, it calls an idle routine, usually called “pause”, “wait”, “yield”, “nop” (stands for no operation), etc.The advantages and disadvantages are very similar to the control loop, except that adding new software is easier, by simply writing a new task, or adding to the queue-interpreter.

- Primitive Multitasking:

In this type of system, a low-level piece of code switches between tasks or threads based on a timer (connected to an interrupt). This is the level at which the system is generally considered to have an "operating system" kernel. Depending on how much

functionality is required, it introduces more or less of the complexities of managing multiple tasks running conceptually in parallel.

As any code can potentially damage the data of another task (except in larger systems using an MMU) programs must be carefully designed and tested, and access to shared data must be controlled by some synchronization strategy, such as message queues, semaphores or a non-blocking synchronization scheme.

Because of these complexities, it is common for organizations to buy a real-time operating system, allowing the application programmers to concentrate on device functionality rather than operating system services, at least for large systems; smaller systems often cannot afford the overhead associated with a generic real time system, due to limitations regarding memory size, performance, and/or battery life.

- Microkernels And Exokernels:

A microkernel is a logical step up from a real-time OS. The usual arrangement is that the operating system kernel allocates memory and switches the CPU to different threads of execution. User mode processes implement major functions such as file systems, network interfaces, etc.

In general, microkernels succeed when the task switching and intertask communication is fast, and fail when they are slow. Exokernels communicate efficiently by normal subroutine calls. The hardware and all the software in the system are available to, and extensible by application programmers. Based on performance, functionality, requirement the embedded systems are divided into three categories:

5.3.2 Stand Alone Embedded System:

These systems takes the input in the form of electrical signals from transducers or commands from human beings such as pressing of a button etc.., process them and produces desired output. This entire process of taking input, processing it and giving output is done in standalone mode. Such embedded systems comes under stand alone embedded systems

Eg: microwave oven, air conditioner etc..

5.3.3 Real-time embedded systems:

Embedded systems which are used to perform a specific task or operation in a specific time period those systems are called as real-time embedded systems. There are two types of real-time embedded systems.

- Hard Real-time embedded systems:

These embedded systems follow an absolute dead line time period i.e.., if the tasking is not done in a particular time period then there is a cause of damage to the entire equipment.

Eg: consider a system in which we have to open a valve within 30 milliseconds. If this valve is not opened in 30 ms this may cause damage to the entire equipment. So in such cases we use embedded systems for doing automatic operations.

- Soft Real Time embedded systems:

These embedded systems follow a relative dead line time period i.e.., if the task is not done in a particular time that will not cause damage to the equipment.

Eg: Consider a TV remote control system , if the remote control takes a few milliseconds delay it will not cause damage either to the TV or to the remote control. These systems which will not cause damage when they are not operated at considerable time period those systems comes under soft real-time embedded systems.

5.3.4 Network communication embedded systems:

A wide range network interfacing communication is provided by using embedded systems.

Eg:

- Consider a web camera that is connected to the computer with internet can be used to spread communication like sending pictures, images, videos etc.., to another computer with internet connection throughout anywhere in the world.
- Consider a web camera that is connected at the door lock.

Whenever a person comes near the door, it captures the image of a person and sends to the desktop of your computer which is connected to internet. This gives an alerting message with image on to the desktop of your computer, and then you can open the door lock just by clicking the mouse. Fig: 5.2 show the network communications in embedded systems.

Fig 5.2: Network communication embedded systems

5.3.5 Different types of processing units:

The central processing unit (c.p.u) can be any one of the following microprocessor, microcontroller, digital signal processing.

- Among these Microcontroller is of low cost processor and one of the main advantage of microcontrollers is, the components such as memory, serial communication interfaces, analog to digital converters etc.., all these are built on a single chip. The numbers of external components that are connected to it are very less according to the application.
- Microprocessors are more powerful than microcontrollers. They are used in major applications with a number of tasking requirements. But the microprocessor requires many external components like memory, serial communication, hard disk, input output ports etc.., so the power consumption is also very high when compared to microcontrollers.
- Digital signal processing is used mainly for the applications that particularly involved with processing of signals

5.4 APPLICATIONS OF EMBEDDED SYSTEMS:

5.4.1 Consumer applications:

At home we use a number of embedded systems which include microwave oven, remote control, vcd players, dvd players, camera etc….

Fig5.3: Automatic coffee makes equipment

5.4.2 Office automation:

We use systems like fax machine, modem, printer etc…

Fig5.4: Fax machine

Fig5.5: Printing machine

5.4.3. Industrial automation:

Today a lot of industries are using embedded systems for process control. In industries we design the embedded systems to perform a specific operation like monitoring temperature, pressure, humidity ,voltage, current etc.., and basing on these monitored levels we do control other devices, we can send information to a centralized monitoring station.

Fig5.6: Robot

In critical industries where human presence is avoided there we can use robots which are programmed to do a specific operation.

5.4.5 Computer networking:

Embedded systems are used as bridges routers etc..

Fig5.7: Computer networking

5.4.6 Tele communications:

Cell phones, web cameras etc.

Fig5.8: Cell Phone

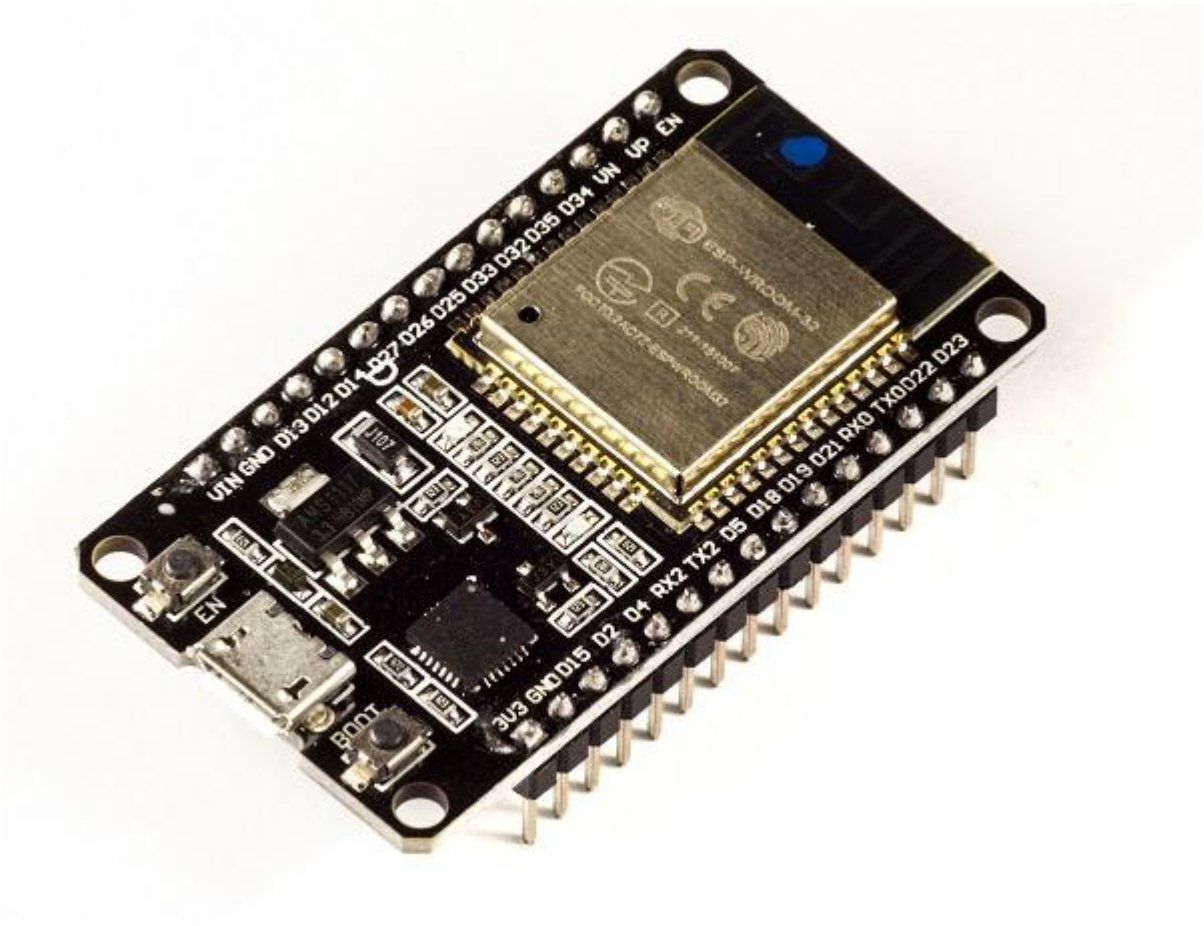
Fig5.9: Web camera

CHAPTER 6

HARDWARE DESCRIPTION

ESP32 CONTROLLER

ESP32 is the SoC (System on Chip) microcontroller which has gained massive popularity recently. Whether the popularity of ESP32 grew because of the growth of IoT or whether IoT grew because of the introduction of ESP32 is debatable. If you know 10 people who have been part of the firmware development for any IoT device, chances are that 7−8 of them would have worked on ESP32 at some point.



Before we delve into the actual reasons for the popularity of ESP32, let's take a look at some of its important specifications. The specs listed below belong to the ESP32 WROOM 32 variant.−

- Integrated Crystal− 40 MHz
- Module Interfaces− UART, SPI, I2C, PWM, ADC, DAC, GPIO, pulse counter, capacitive touch sensor
- Integrated SPI flash− 4 MB
- ROM− 448 KB (for booting and core functions)
- SRAM− 520 KB
- Integrated Connectivity Protocols− WiFi, Bluetooth, BLE
- On−chip sensor− Hall sensor
- Operating temperature range− −40 − 85 degrees Celsius
- Operating Voltage− 3.3V
- Operating Current− 80 mA (average)

With the above specifications in front of you, it is very easy to decipher the reasons for ESP32's popularity. Consider the requirements an IoT device would have from its microcontroller (μC). If you've gone through the previous chapter, you'd have realized that the major operational blocks of any IoT device are sensing, processing, storage, and transmitting. Therefore, to begin with, the μC should be able to interface with a variety of sensors. It should support all the common communication protocols required for sensor interface: UART, I2C, SPI. It should have ADC and pulse counting capabilities. ESP32 fulfills all of these requirements. On top of that, it also can interface with capacitive touch sensors. Therefore, most common sensors can interface seamlessly with ESP32.

Secondly, the μC should be able to perform basic processing of the incoming sensor data, sometimes at high speeds, and have sufficient memory to store the data. ESP32 has a max operating frequency of 40 MHz, which is sufficiently high. It has two cores, allowing parallel processing, which is a further add-on. Finally, its 520 KB SRAM is sufficiently large for processing a large array of data onboard. Many popular processes and transforms, like FFT, peak detection, RMS calculation, etc. can be performed onboard ESP32. On the storage front, ESP32 goes a step ahead of the conventional microcontrollers and provides a file system within the flash. Out of the 4 MB of onboard flash, by default, 1.5 MB is reserved as SPIFFS (SPI Flash File System). Think of it as a mini−SD Card that lies within the chip itself. You can not only store data, but also text files, images, HTML and CSS files, and a lot more within SPIFFS. People have displayed beautiful WebPages on WiFi servers created using ESP32, by storing HTML files within SPIFFS.

Finally, for transmitting data, ESP32 has integrated WiFi and Bluetooth stacks, which have proven to be a game-changer. No need to connect a separate module (like a GSM module or an LTE module) for testing cloud communication. Just have the ESP32 board and a running WiFi, and you can get started. ESP32 allows you to use WiFi in Access Point as well as Station Mode. While it supports TCP/IP, HTTP, MQTT, and other traditional communication protocols, it also supports HTTPS. Yep, you heard that right. It has a crypto−core or a crypto-accelerator, a dedicated piece of hardware whose job is to accelerate the encryption process. So you cannot only communicate with your web server, you can do so securely. BLE support is also critical for several applications. Of course, you can interface LTE or GSM or LoRa modules with ESP32. Therefore, on the 'transmitting data' front as well, ESP32 exceeds expectations.

ESP32 is a chip that provides Wi-Fi and (in some models) Bluetooth connectivity for embedded devices – in other words, for IoT devices. While ESP32 is technically just the chip,

the modules and development boards that contain this chip are often also referred to as "ESP32" by the manufacturer.

The original ESP32 chip had a single core Tensilica Xtensa LX6 microprocessor. The processor had a clock rate of over 240 MHz, which made for a relatively high data processing speed.

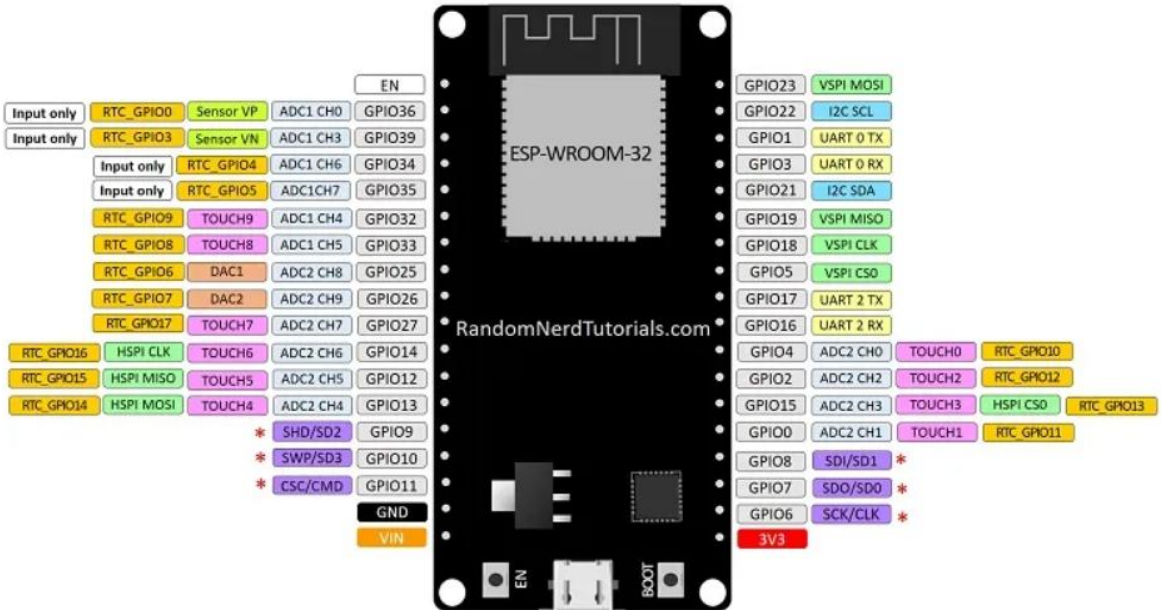
More recently, new models were added, including the ESP32-C and -S series, which include both single and dual core variations. These two series also rely on a Risc-V CPU model instead of Xtensa. Risc-V is similar to the ARM architecture, which is well-supported and well-known, but Risc-V is open source and easy to use. Specifically, Risc-V and ARM have good support from GNU compilers, while the Xtensa needed extra support and development to work with the compilers.

The newer models are available with combined Wi-Fi and Bluetooth connectivity, or just Wi-Fi connectivity. There are several different chip models available, including:

- ESP32-D0WDQ6 (and ESP32D0WD)
- ESP32-D2WD
- ESP32-S0WD
- System in package (SiP) – ESP32-PICO-D4
- ESP32 S series
- ESP32-C series
- ESP32-H series

The ESP32 is most commonly used for mobile devices, wearable tech, and IoT applications, such as Nabto Edge. Moreover, since Mongoose OS introduced an ESP32 IoT Starter Kit, the ESP32 has gained a reputation as the ultimate chip for hobbyists and IoT developers. It's suitable for commercial IoT, and its capabilities and resources have grown impressively over the past four years.

ESP32 features and specifications



Here's a high-level summary of the features and specifications of the ESP32:

ESP-32	DESCRIPTION
Core	2
Architecture	32 bits
Clock	Tensilica Xtensa LX106 160-240MHz
WiFi	IEEE802.11 b/g/n
Bluetooth	Yes - classic & BLE
RAM	520KB
Flash	External QSPI - 16MB
GPIO	22
DAC	2
ADC	18
Interfaces	SPI-I2C-UART-I2S-CAN

- Processors – The ESP32 uses a Tensilica Xtensa 32-bit LX6 microprocessor. This typically relies on a dual core architecture, with the exception of one module, the ESP32-S0WD, which uses a single-core system. The clock frequency reaches up to 240MHz and it performs up to 600 DMIPS (Dhrystone millions of instructions per second). Moreover, its low power consumption allows for analog to digital conversions as well as computation and level thresholds, even while the chip is in deep sleep mode.

- Wireless connectivity – The ESP32 enables connectivity to integrated Wi-Fi through the 802.11 b/g/n/e/i/. Moreover, Bluetooth connectivity is made possible with the v4.2 BR/EDR, and the series also features Bluetooth low energy (BLE).
- Memory – Internal memory for the ESP32 is as follows. ROM: 448 KB (for booting/core functions), SRAM: 520 KB (for data/instructions), RTC fast SRAM: 8 KB (for data storage/main CPU during boot from sleep mode), RTC slow SRAM: 8 KB (for co-processor access during sleep mode), and eFuse: 1 KiBit (256 bits used for the system (MAC address and chip configuration) and 768 bits reserved for customer applications). Moreover, some of the ESP32 chips, including the ESP32-D2WD and ESP32-PICO-D4, have internally connected flash. See the respective internal flash memory for each chip in the ESP32 Chips section.
- External flash and SRAM – ESP32 supports up to four 16 MB external QSPI flashes and SRAMs with hardware encryption based on AES to protect developers' programs and data. It accesses the external QSPI flash and SRAM through high-speed caches.
- Security – The ESP32 supports all IEEE 802.11 standard security features, including WFA, WPA/WPA2 and WAPI. Moreover, ESP32 has a secure boot and flash encryption.

ESP32 functions

ESP32 has many applications when it comes to IoT. Here are just some of the IoT functions the chip is used for:

- Networking: The module's Wi-Fi antenna and dual core enables embedded devices to connect to routers and transmit data.
- Data processing: Includes processing basic inputs from analog and digital sensors to far more complex calculations with an RTOS or non-OS software development kit (SDK). A non-OS SDK refers to one that is designed to run directly on the chip without a full operating system supporting it.
- P2P connectivity: Creates direct communication between different ESPs and other devices using IoT P2P connectivity.
- Web server: Provides access to pages written in HTML or development languages.

ESP32 Applications

The ESP32 modules are commonly found in the following IoT devices:

- Smart industrial devices, including programmable logic controllers (PLCs)
- Smart medical devices, including wearable health monitors
- Smart energy devices, including HVAC and thermostats
- Smart security devices, including surveillance cameras and smart locks

Chip versus modules versus development boards

The ESP32 is just the name of the chip. Device manufacturers and developers have three different format choices, and the decision of which one to go with will depend on their individual circumstances:

- ESP32 chip: This is the bare-bones chip that is manufactured by Espressif. It comes unshielded, meaning there's no protective casing, and it can't be attached to a module or board without soldering. Therefore, most device manufacturers do not purchase just the chip, as this will add an additional layer of complexity to the production process.
- ESP32 modules: These are surface mountable modules that contain the chip. The modules are essentially small electrical components that can be attached to a circuit board. The benefit here is that you can easily mount these modules onto an MCU board. The chip is also usually shielded and pre-approved by the FCC, which means device manufacturers do not need to worry about adding additional steps to the production process to achieve FCC compliance regarding Wi-Fi shielding.
- ESP32 development boards: These are IoT MCU development boards that have the modules containing the ESP32 chip preinstalled. They are used by hobbyists, device manufacturers and developers to test and prototype IoT devices before entering mass production. There is a wide variety of makes and models of ESP32 development boards, produced by different manufacturers. Here are some important specs to consider when choosing a suitable IoT ESP32 development board:
    - GPIO pins
    - ADC pins
    - Wi-Fi antennas
    - LEDs
    - Shielding*

- Flash memory

*Many international markets require shielded Wi-Fi devices, as Wi-Fi produces a lot of radio frequency interference (RFI), and shielding minimizes this interference. This should, therefore, be a key consideration for all developers and embedded device manufacturers.

6.6 Regulated Power Supply:

A regulated power supply converts unregulated AC (Alternating Current) to a constant DC (Direct Current). A regulated power supply is used to ensure that the output remains constant even if the input changes. A regulated DC power supply is also known as a linear power supply; it is an embedded circuit and consists of various blocks. The regulated power supply will accept an AC input and give a constant DC output. The figure below shows the block diagram of a typical regulated DC power supply.

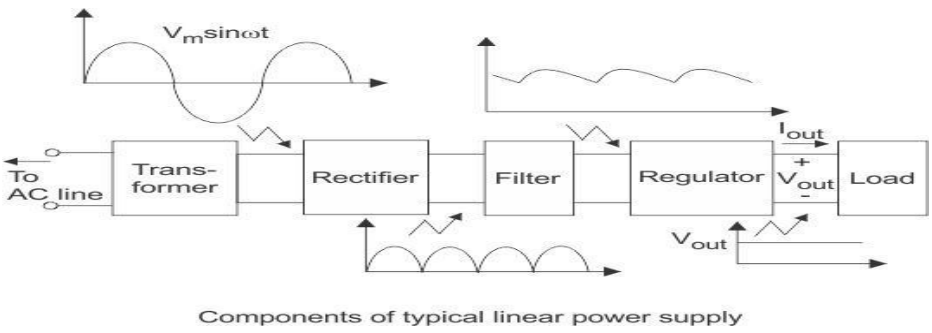


Figure 6.8 Block diagram of power supply

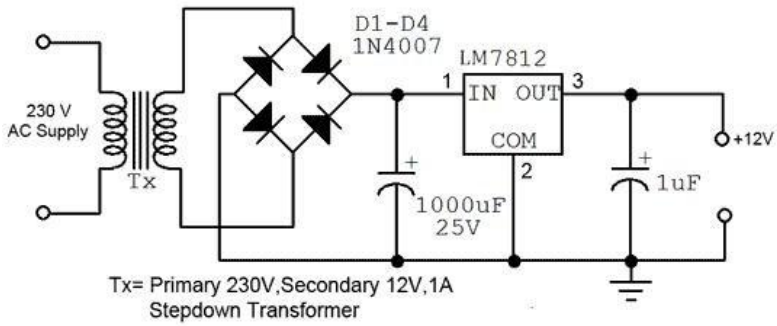


Figure 6.9 Power supply circuit diagram

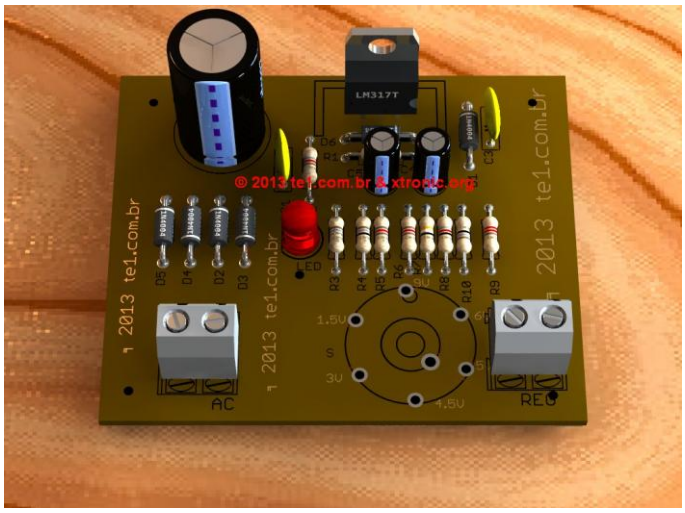


Fig6.10 Regulated Power Supply

The basic circuit diagram of a regulated power supply (DC O/P) with led connected as load is shown in fig: 3.3.3.

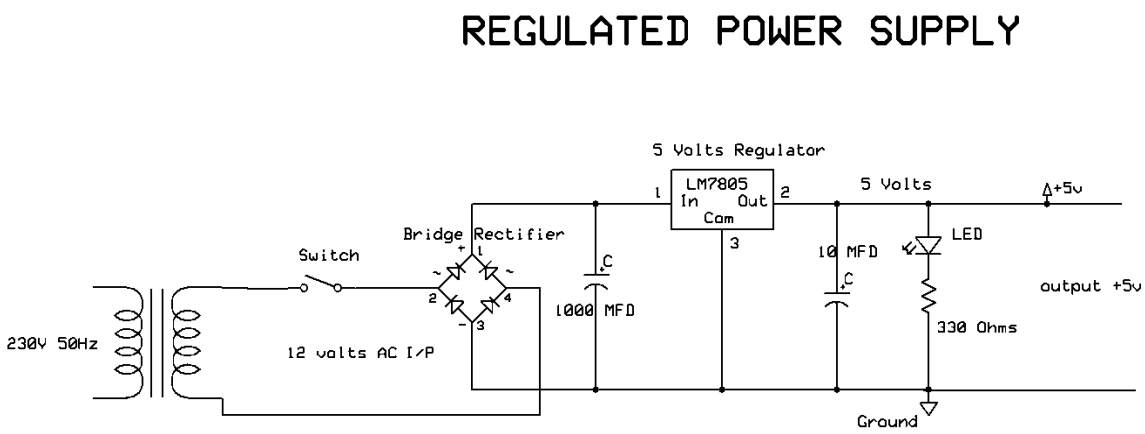


Fig 6.11 Circuit diagram of Regulated Power Supply with Led connection

The components mainly used in above figure are

- 230V AC MAINS
- TRANSFORMER
- BRIDGE RECTIFIER(DIODES)
- CAPACITOR
- VOLTAGE REGULATOR(IC 7805)

- RESISTOR
- LED(LIGHT EMITTING DIODE)

The detailed explanation of each and every component mentioned above is as follows:

Transformation: The process of transforming energy from one device to another is called transformation. For transforming energy we use transformers.

Step down transformer:

Incase of step down transformer, Primary winding induces more flux than the secondary winding, and secondary winding is having less number of turns because of that it accepts less number of flux, and releases less amount of voltage.

Bridge full wave rectifier:

The Bridge rectifier circuit is shown in fig:3.8, which converts an ac voltage to dc voltage using both half cycles of the input ac voltage. The Bridge rectifier circuit is shown in the figure. The circuit has four diodes connected to form a bridge. The ac input voltage is applied to the diagonally opposite ends of the bridge. The load resistance is connected between the other two ends of the bridge.

For the positive half cycle of the input ac voltage, diodes D1 and D3 conduct, whereas diodes D2 and D4 remain in the OFF state. The conducting diodes will be in series with the load resistance R_L and hence the load current flows through R_L.

For the negative half cycle of the input ac voltage, diodes D2 and D4 conduct whereas, D1 and D3 remain OFF. The conducting diodes D2 and D4 will be in series with the load resistance R_L and hence the current flows through R_L in the same direction as in the previous half cycle. Thus a bi-directional wave is converted into a unidirectional wave.

Input Output

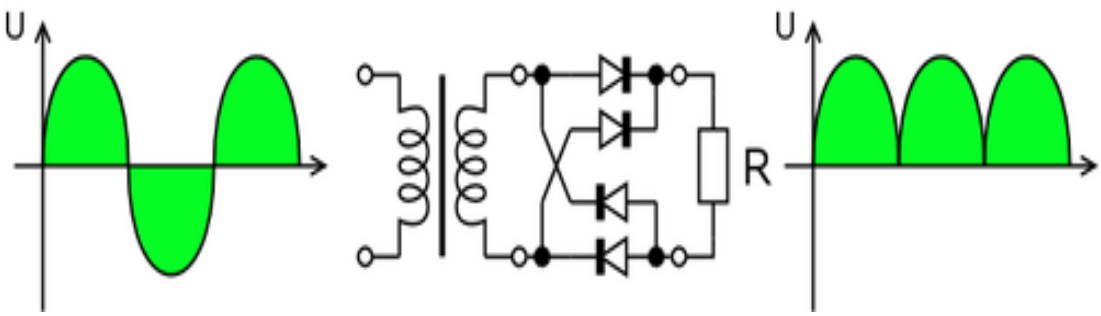


Fig 6.12 Bridge rectifier: a full-wave rectifier using 4 diodes

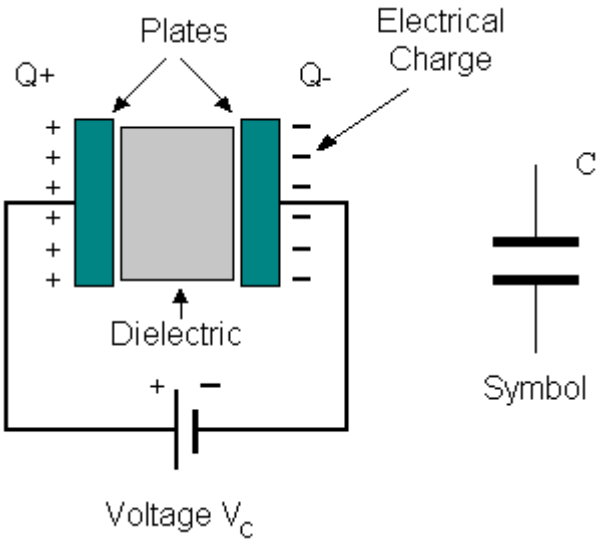
DB107:

Now -a -days Bridge rectifier is available in IC with a number of DB107. In our project we are using an IC in place of bridge rectifier. The picture of DB 107 is shown in fig: 3.3.8.

Fig 6.13. DB107

Capacitor Filtration:

The process of converting a pulsating direct current to a pure direct current using filters is called as filtration.Electronic filters are electronic circuits, which perform signal-processing functions, specifically to remove unwanted frequency components from the signal, to enhance wanted ones.The Capacitor or sometimes referred to as a Condenser is a passive device, and one which stores energy in the form of an electrostatic field which produces a potential (static voltage) across its plates. In its basic form a capacitor consists of two parallel conductive plates that are not connected but are electrically separated either by air or by an insulating material called the Dielectric. When a voltage is applied to these plates, a current flows charging up the plates with electrons giving one plate a positive charge and the other plate an equal and opposite negative charge this flow of electrons to the plates is known as the Charging Current and continues to flow until the voltage across the plates (and hence the capacitor) is equal to the applied voltage Vcc. At this point the capacitor is said to be fully charged and this is illustrated below. The construction of capacitor and an electrolytic capacitor are shown in figures 3.3.9 and 3.3.10 respectively.



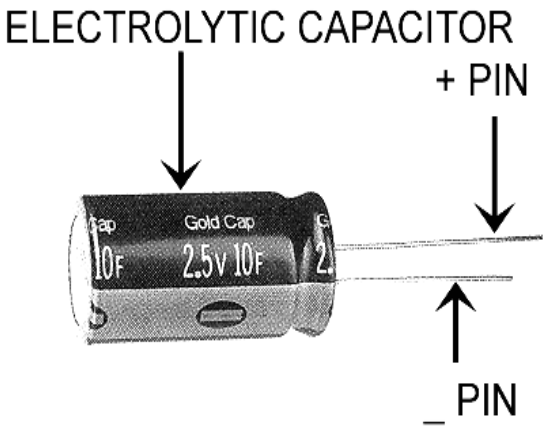


Fig 6.14 Construction Of a Capacitor

Fig 6.15 Electrolytic

Voltage Regulation:

The process of converting a varying voltage to a constant regulated voltage is called as regulation. For the process of regulation we use voltage regulators.

Voltage Regulator:

A voltage regulator (also called a 'regulator') with only three terminals appears to be a simple device, but it is in fact a very complex integrated circuit. It converts a varying input voltage into a constant 'regulated' output voltage. Voltage Regulators are available in a variety of outputs like 5V, 6V, 9V, 12V and 15V. The LM78XX series of voltage regulators are designed for positive input. For applications requiring negative input,

the LM79XX series is used. Using a pair of ‘voltage-divider’ resistors can increase the output voltage of a regulator circuit.

It is not possible to obtain a voltage lower than the stated rating. You cannot use a 12V regulator to make a 5V power supply. Voltage regulators are very robust. These can withstand over-current draw due to short circuits and also over-heating. In both cases, the regulator will cut off before any damage occurs. The only way to destroy a regulator is to apply reverse voltage to its input. Reverse polarity destroys the regulator almost instantly. Fig: 3.3.11 shows voltage regulator.

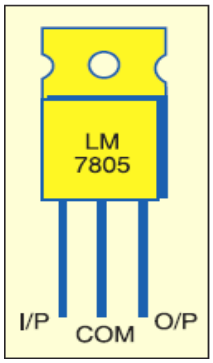


Fig 6.16. Voltage Regulator

Resistor:

A resistor is a two-terminal electronic component that produces a voltage across its terminals that is proportional to the electric current passing through it in accordance with Ohm's law:

$V = IR$

Resistors are elements of electrical networks and electronic circuits and are ubiquitous in most electronic equipment. Practical resistors can be made of various compounds and films, as well as resistance wire (wire made of a high-resistivity alloy, such as nickel/chrome).

The primary characteristics of a resistor are the resistance, the tolerance, maximum working voltage and the power rating. Other characteristics include temperature coefficient, noise, and inductance. Less well-known is critical resistance, the value below which power dissipation limits the maximum permitted current flow, and above which the limit is applied voltage. Critical resistance is determined by the design, materials and dimensions of the resistor.

dissipated by a resistor (or the equivalent resistance of a resistor network) is calculated using the following:

$$P = I^2R = IV = \frac{V^2}{R}$$

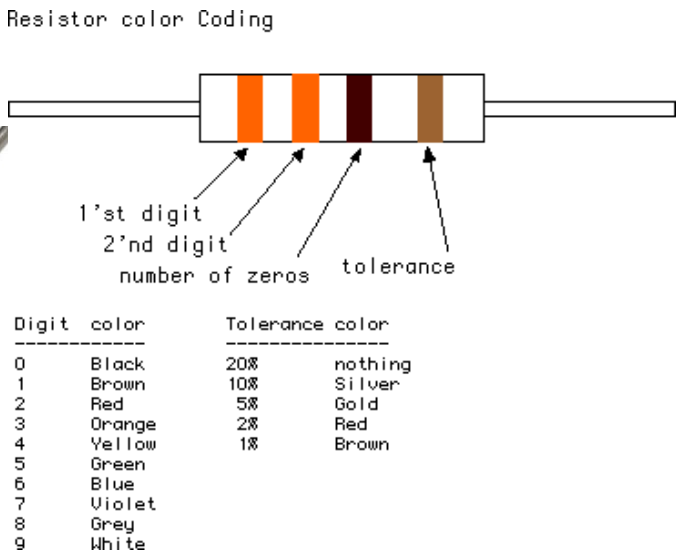
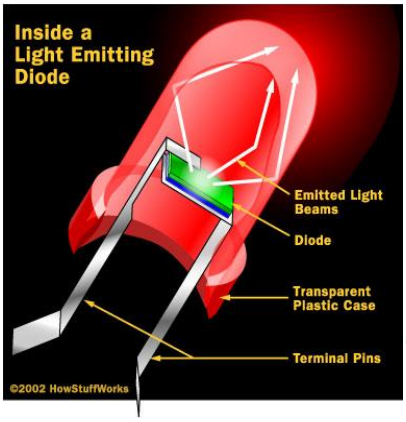


Fig 6.18 Resistor

Fig 6.19 Color Bands In Resistor

LED:

A light-emitting diode (LED) is a semiconductor light source. LED's are used as indicator lamps in many devices, and are increasingly used for lighting. Introduced as a practical electronic component in 1962, early LED's emitted low-intensity red light, but modern versions are available across the visible, ultraviolet and infrared wavelengths, with very high brightness. The internal structure and parts of a led are shown in figures 3.4.1 and 3.4.2 respectively.



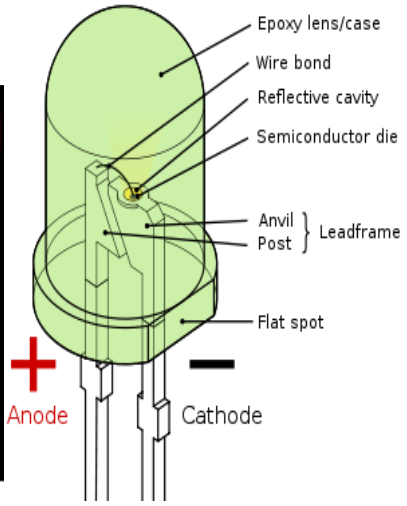


Fig.6.20 Inside a LED

Fig 6.21. Parts of a LED

6.9 Relay:

A relay is an electrically operated switch. Many relays use an electromagnet to operate a switching mechanism, but other operating principles are also used. Relays find applications where it is necessary to control a circuit by a low-power signal, or where several circuits must be controlled by one signal. The first relays were used in long distance telegraph circuits, repeating the signal coming in from one circuit and re-transmitting it to another. Relays found extensive use in telephone exchanges and early computers to perform logical operations. A type of relay that can handle the high power required to directly drive an electric motor is called a contactor. Solid-state relays control power circuits with no moving parts, instead using a semiconductor device triggered by light to perform switching. Relays with calibrated operating characteristics and sometimes multiple operating coils are used to protect electrical circuits from overload or faults; in modern electric power systems these functions are performed by digital instruments still called "protection relays".

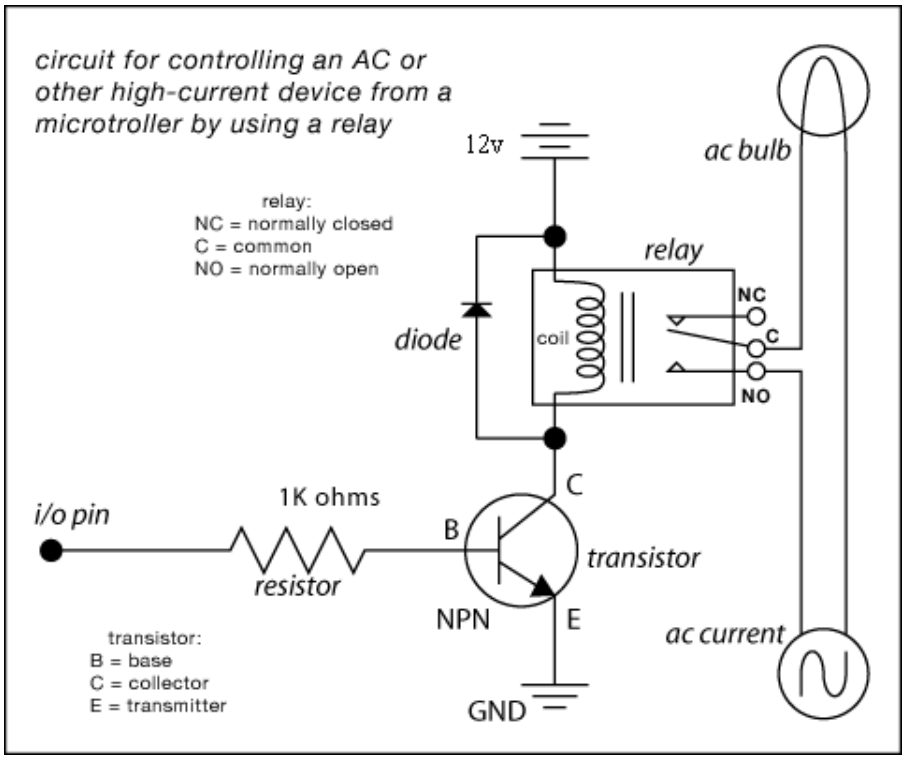


Fig :6.23 Relay circuit

Relay Driver:

The current needed to operate the relay coil is more than can be supplied by most chips (op. amps etc), so a transistor is usually needed, as shown in the diagram below.

Use BC109C or similar. A resistor of about 4k7 will probably be alright. The diode is needed to short circuit the high voltage “back emf” induced when current flowing through the coil is suddenly switched off.

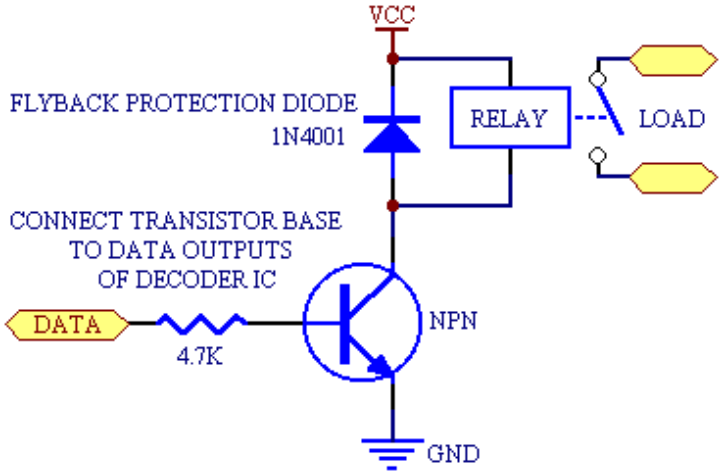


Fig: 6.24 Relay Driver

6.10 LCD DISPLAY

LCD Background:

One of the most common devices attached to a micro controller is an LCD display. Some of the most common LCD’s connected to the many microcontrollers are 16x2 and 20x2 displays. This means 16 characters per line by 2 lines and 20 characters per line by 2 lines, respectively.

Basic 16x 2 Characters LCD

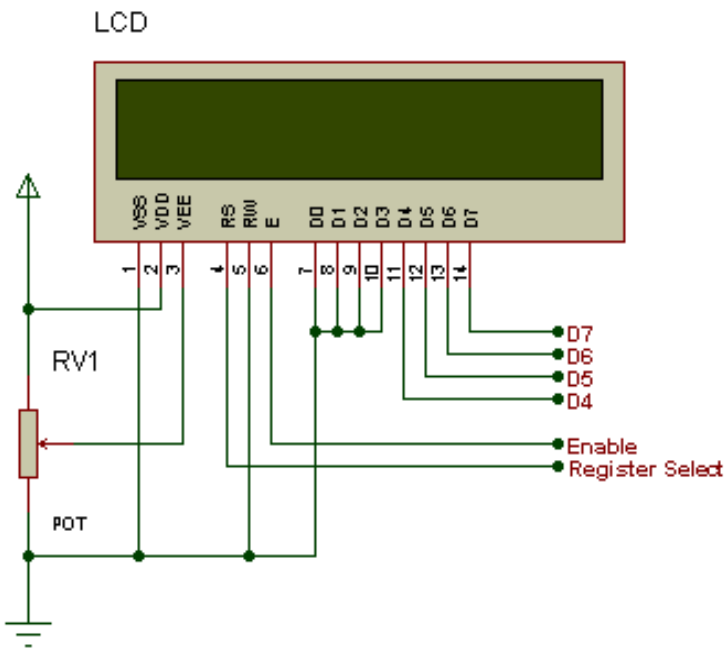


Figure 6.25 LCD Pin diagram

Pin description:

Pin No.	Name	Description
Pin no. 1	VSS	Power supply (GND)
Pin no. 2	VCC	Power supply (+5V)
Pin no. 3	VEE	Contrast adjust
Pin no. 4	RS	0 = Instruction input 1 = Data input
Pin no. 5	R/W	0 = Write to LCD module 1 = Read from LCD module
Pin no. 6	EN	Enable signal
Pin no. 7	D0	Data bus line 0 (LSB)
Pin no. 8	D1	Data bus line 1

Pin no. 9	D2	Data bus line 2
Pin no. 10	D3	Data bus line 3
Pin no. 11	D4	Data bus line 4
Pin no. 12	D5	Data bus line 5
Pin no. 13	D6	Data bus line 6
Pin no. 14	D7	Data bus line 7 (MSB)

Table 1: Character LCD pins with Microcontroller

The LCD requires 3 control lines as well as either 4 or 8 I/O lines for the data bus. The user may select whether the LCD is to operate with a 4-bit data bus or an 8-bit data bus. If a 4-bit data bus is used the LCD will require a total of 7 data lines (3 control lines plus the 4 lines for the data bus). If an 8-bit data bus is used the LCD will require a total of 11 data lines (3 control lines plus the 8 lines for the data bus).

The three control lines are referred to as EN, RS, and RW.

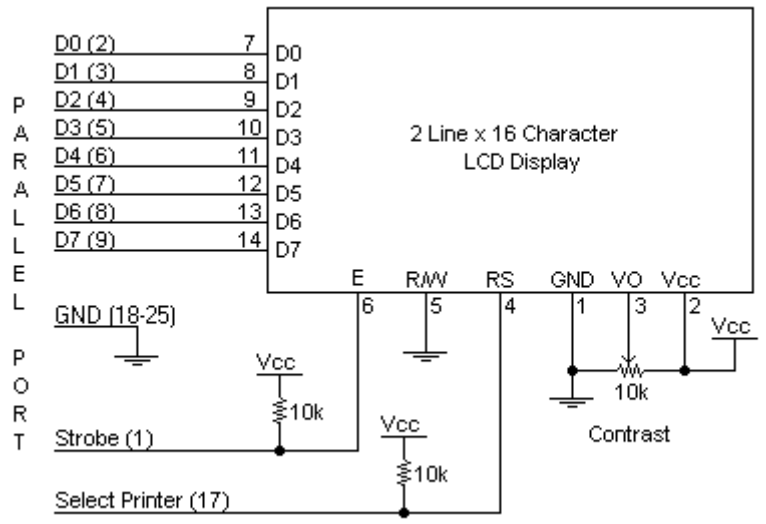
The EN line is called "Enable." This control line is used to tell the LCD that we are sending it data. To send data to the LCD, our program should make sure this line is low (0) and then set the other two control lines and/or put data on the data bus. When the other lines are completely ready, bring EN high (1) and wait for the minimum amount of time required by the LCD datasheet (this varies from LCD to LCD), and end by bringing it low (0) again.

The RS line is the "Register Select" line. When RS is low (0), the data is to be treated as a command or special instruction (such as clear screen, position cursor, etc.). When RS is high (1), the data being sent is text data which should be displayed on the screen. For example, to display the letter "T" on the screen we would set RS high.

The RW line is the "Read/Write" control line. When RW is low (0), the information on the data bus is being written to the LCD. When RW is high (1), the program is effectively querying (or reading) the LCD. Only one instruction ("Get LCD status") is a read command. All others are write commands--so RW will almost always be low.

Finally, the data bus consists of 4 or 8 lines (depending on the mode of operation selected by the user). In the case of an 8-bit data bus, the lines are referred to as DB0, DB1, DB2, DB3, DB4, DB5, DB6, and DB7.

Schematic:



Circuit Description:

Above is the quite simple schematic. The LCD panel's Enable and RegisterSelect is connected to the Control Port. The Control Port is an open collector / open drain output. While most Parallel Ports have internal pull-up resistors, there is a few which don't. Therefore by incorporating the two 10K external pull up resistors, the circuit is more portable for a wider range of computers, some of which may have no internal pull up resistors.

We make no effort to place the Data bus into reverse direction. Therefore we hard wire the R/W line of the LCD panel, into write mode. This will cause no bus conflicts on the data lines. As a result we cannot read back the LCD's internal Busy Flag which tells us if the LCD has accepted and finished processing the last instruction. This problem is overcome by inserting known delays into our program.

The 10k Potentiometer controls the contrast of the LCD panel. Nothing fancy here. As with all the examples, I've left the power supply out. We can use a bench power supply set to 5v or use an onboard +5 regulator. Remember a few de-coupling capacitors, especially if we have trouble with the circuit working properly.

SETB RW

Handling the EN control line:

As we mentioned above, the EN line is used to tell the LCD that we are ready for it to execute an instruction that we've prepared on the data bus and on the other control lines. Note that the EN line must be raised/ lowered before/after each instruction sent to the LCD regardless of whether that instruction is read or write text or instruction. In short, we must always manipulate EN when communicating with the LCD. EN is the LCD's way of knowing that we are talking to it. If we don't raise/lower EN, the LCD doesn't know we're talking to it on the other lines.

Thus, before we interact in any way with the LCD we will always bring the EN line low with the following instruction:

CLR EN

And once we've finished setting up our instruction with the other control lines and data bus lines, we'll always bring this line high:

SETB EN

The line must be left high for the amount of time required by the LCD as specified in its datasheet. This is normally on the order of about 250 nanoseconds, but checks the datasheet. In the case of a typical microcontroller running at 12 MHz, an instruction requires 1.08 microseconds to execute so the EN line can be brought low the very next instruction. However, faster microcontrollers (such as the DS89C420 which executes an instruction in 90 nanoseconds given an 11.0592 MHz crystal) will require a number of NOPs to create a delay while EN is held high. The number of NOPs that must be inserted depends on the microcontroller we are using and the crystal we have selected.

The instruction is executed by the LCD at the moment the EN line is brought low with a final CLR EN instruction.

Checking the busy status of the LCD:

As previously mentioned, it takes a certain amount of time for each instruction to be executed by the LCD. The delay varies depending on the frequency of the crystal attached to the oscillator input of the LCD as well as the instruction which is being executed.

While it is possible to write code that waits for a specific amount of time to allow the LCD to execute instructions, this method of "waiting" is not very flexible. If the crystal

frequency is changed, the software will need to be modified. A more robust method of programming is to use the "Get LCD Status" command to determine whether the LCD is still busy executing the last instruction received.

The "Get LCD Status" command will return to us two tidbits of information; the information that is useful to us right now is found in DB7. In summary, when we issue the "Get LCD Status" command the LCD will immediately raise DB7 if it's still busy executing a command or lower DB7 to indicate that the LCD is no longer occupied. Thus our program can query the LCD until DB7 goes low, indicating the LCD is no longer busy. At that point we are free to continue and send the next command.

6.13. Energy Meter

This Energy meter module is measure the energy consumption by home electrical appliances. In this internally voltage and current sensors inbuilt for measuring of volt and current then onwards to measure power and finally energy. This is LCD and LED indicating energy meter operated with 240v AC supply, 50Hz frequency and compatibility of 10A current usage. When unit consumption done it indicated by led then we can capture that signal using LDR sensor and automatically data sends to microcontroller.

Figure 5: Energy Meter Module

3.5 ESP8266 MODULE

1 Introduction

Espressif Systems' Smart Connectivity Platform (ESCP) of high performance wireless SOCs, for mobile platform designers, provides unsurpassed ability to embed Wi-Fi capabilities within other systems, at the lowest cost with the greatest functionality

2 Technology Overview

ESP8266 offers a complete and self-contained Wi-Fi networking solution, allowing it to either host the application or to offload all Wi-Fi networking functions from another application processor.

When ESP8266 hosts the application, and when it is the only application processor in the device, it is able to boot up directly from an external flash. It has integrated cache to improve the performance of the system in such applications, and to minimize the memory requirements.

Alternately, serving as a Wi-Fi adapter, wireless internet access can be added to any microcontroller-based design with simple connectivity through UART interface or the CPU AHB bridge interface.

ESP8266 on-board processing and storage capabilities allow it to be integrated with the sensors and other application specific devices through its GPIOs with minimal development up-front and minimal loading during runtime. With its high degree of on-chip integration, which includes the antenna switch balun, power management converters, it requires minimal external circuitry, and the entire solution, including front-end module, is designed to occupy minimal PCB area.

Sophisticated system-level features include fast sleep/wake context switching for energy-efficient VoIP, adaptive radio biasing for low-power operation, advance signal processing, and spur cancellation and radio co-existence features for common cellular, Bluetooth, DDR, LVDS, LCD interference mitigation.

3 Features

- 802.11 b/g/n protocol
- Wi-Fi Direct (P2P), soft-AP
- Integrated TCP/IP protocol stack
- Integrated TR switch, balun, LNA, power amplifier and matching network
- Integrated PLL, regulators, and power management units
- +19.5dBm output power in 802.11b mode
- Integrated temperature sensor
- Supports antenna diversity
- Power down leakage current of < 10uA
- Integrated low power 32-bit CPU could be used as application processor
- SDIO 2.0, SPI, UART
- STBC, 1×1 MIMO, 2×1 MIMO
- A-MPDU & A-MSDU aggregation & 0.4s guard interval
- Wake up and transmit packets in < 2ms
- Standby power consumption of < 1.0mW (DTIM3)

ESP8266 Applications

- Smart power plugs
- Home automation
- Mesh network
- Industrial wireless control
- Baby monitors
- IP Cameras
- Sensor networks
- Wearable electronics
- Wi-Fi location-aware devices
- Security ID tags
- Wi-Fi position system beacons

Specifications

7.1 Current Consumption

The following current consumption is based on 3.3V supply, and 25□C ambient, using internal regulators. Measurements are done at antenna port without SAW filter. All the transmitter's measurements are based on 90% duty cycle, continuous transmit mode.

Mode		**Typ**	**Unit**
Transmit 802.11b, CCK 1Mbps, POUT=+19.5dBm		215	mA
Transmit 802.11b, CCK 11Mbps, POUT=+18.5dBm		197	mA
Transmit 802.11g, OFDM 54Mbps, POUT =+16dBm		145	mA
Transmit 802.11n, MCS7, POUT=+14dBm		135	mA
Receive 802.11b, packet length=1024 byte, -80dBm		60	mA
Receive 802.11g, packet length=1024 byte, -70dBm		60	mA
Receive 802.11n, packet length=1024 byte, -65dBm		62	mA
Standby		0.9	mA

Deep sleep	10	uA
Power save mode DTIM 1	1.2	mA
Power save mode DTIM 3	0.86	mA
Total shutdown	0.5	uA

CPU, Memory and Interfaces

CPU

This chip embeds an ultra low power Micro 32-bit CPU, with 16-bit thumb mode. This CPU can be interfaced using:

- code RAM/ROM interface (iBus) that goes to the memory controller, that can also be used to access external flash memory,
- data RAM interface (dBus), that also goes to the memory controller
- AHB interface, for register access, and
- JTAG interface for debugging

Memory Controller

The memory controller contains ROM, and SRAM. It is accessed by the CPU using the iBus, dBus and AHB interface. Any of these interfaces can request access to the ROM or RAM modules, and the memory controller arbiters serve these 3 interfaces on a first-come-first-serve basis.

AHB and AHB Blocks

The AHB blocks performs the function of an arbiter, controls the AHB interfaces from the MAC, SDIO (host) and CPU. Depending on the address, the AHB data requests can go into one of the two slaves:

- APB block, or
- flash controller (usually for standalone applications).

Data requests to the memory controller are usually high speed requests, and requests to the APB block are usually register access.

The APB block acts as a decoder. It is meant only for access to programmable registers within ESP8266's main blocks. Depending on the address, the APB request can go to the radio, SI/SPI, SDIO (host), GPIO, UART, real-time clock (RTC), MAC or digital baseband.

Interfaces

The ESP8266 contains several analog and digital interfaces described in the following sections.

Master SI / SPI Control (Optional)

The master serial interface (SI) can operate in two, three or four-wire bus configurations to control the EEPROM or other I2C/SPI devices. Multiple I2C devices with different device addresses are supported by sharing the 2-wire bus.

Multiple SPI devices are supported by sharing the clock and data signals, using separate software controlled GPIO pins as chip selects.

The SPI can be used for controlling external devices such as serial flash memories, audio CODECs, or other slave devices. It is set up as a standard master SPI device with 3 different enable pins:

- SPI_EN0,
- SPI_EN1,
- SPI_EN2.

Both SPI master and SPI slave are supported with the latter being used as a host interface.

SPI_EN0 is used as an enable signal to an external serial flash memory for downloading patch code and/or MIB-data to the baseband in an embedded application. In a host based application, patch code and MIB-data can alternatively be downloaded via the host interface. This pin is active low and should be left open if not used.

SPI_EN1 is usually used for a user application, e.g. to control an external audio codec or sensor ADC, in an embedded application. This pin is active low and should be left open if not used.

SPI_EN2 usually controls an EEPROM to store individual data, such as MIB information, MAC address, and calibration data, or for general use. This pin is active low and should be left open if not used.

General Purpose IO

There are up to 16 GPIO pins. They can be assigned to various functions by the firmware. Each GPIO can be configured with internal pull-up/down, input available for sampling by a software register, input triggering an edge or level CPU interrupt, input triggering a level wakeup interrupt, open-drain or push-pull output driver, or output source from a software register, or a sigma-delta PWM DAC.

These pins are multiplexed with other functions such as host interface, UART, SI, Bluetooth coexistence, etc.

Digital IO Pads

The digital IO pads are bidirectional, non-inverting and tri-state. It includes input and an output buffer with tristate control inputs. Besides this, for low power operations, the IO can also be set to hold. For instance, when we power down the chip, all output enable signals can be set to hold low. Optional hold functionality can be built into the IO if requested. When the IO is not driven by the internal or external circuitry, the hold functionality can be used to hold the state to the last used state.

The hold functionality introduces some positive feedback into the pad. Hence, the external driver that drives the pad must be stronger than the positive feedback. The required drive strength is however small – in the range of 5uA.

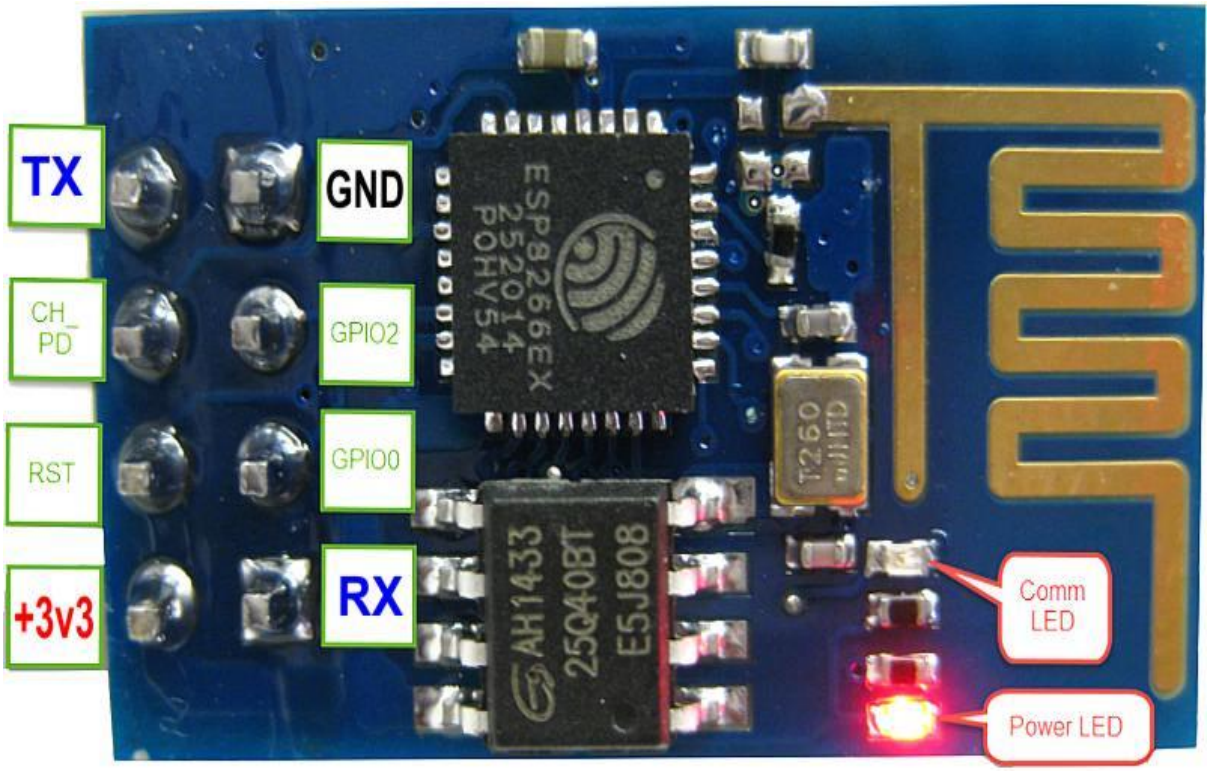
Parameter	**Symbol**	**Min**	**Max**	**Unit**
Input low voltage	**VIL**	**-0.3**	**0.25□VIO**	**V**
Input high voltage	**VIH**	**0.75□VIO**	**3.6**	**V**
Input leakage current		**IIL**	**50**	**nA**

Output low voltage		**VOL**	**0.1□VIO**	**V**
Output high voltage		**VOH**	**0.8□VIO**	**V**
Input pin capacitance		**Cpad**	**2**	**pF**
VDDIO	**VIO**	**1.7**	**3.6**	**V**
Maximum drive capability		**IMAX**	**12**	**mA**
Temperature	**Tamb**	**-20**	**100**	□C

Applications

- **Wi-Fi Smart Hardware Converted from UART Serial Ports**
- **Sensor**
- **Smart Light**
- **Smart Plug**

Wi-Fi (Short for Wireless Fidelity) is a wireless technology that uses radio frequency to transmit data through the air. Wi-Fi has initial speeds of 1mbps to 2mbps. Wi-Fi transmits data in the frequency band of 2.4 GHz. It implements the concept of frequency division multiplexing technology. Range of Wi-Fi technology is 40-300 feet.



2.2 Wi-fi module

ESP8266 offers a complete and self-contained Wi-Fi networking solution, allowing it to either host the application or to offload all Wi-Fi networking functions from another application processor.

When ESP8266 hosts the application, and when it is the only application processor in the device, it is able to boot up directly from an external flash. It has integrated cache to improve the performance of the system in such applications, and to minimize the memory requirements.

Alternately, serving as a Wi-Fi adapter, wireless internet access can be added to any microcontroller-based design with simple connectivity through UART interface or the CPU AHB bridge interface.

ESP8266 on-board processing and storage capabilities allow it to be integrated with the sensors and other application specific devices through its GPIOs with minimal development up-front and minimal loading during runtime. With its high degree of on-chip integration, which includes the antenna switch balun, power management converters, it requires minimal external circuitry, and the entire solution, including front-end module, is designed to occupy minimal PCB area.

Sophisticated system-level features include fast sleep/wake context switching for energy-efficient VoIP, adaptive radio biasing for low-power operation, advance signal processing,

and spur cancellation and radio co-existence features for common cellular, Bluetooth, DDR, LVDS, LCD interference mitigation.

2.2.1FEATURES

- 802.11 b/g/n protocol
- Wi-Fi Direct (P2P), soft-AP
- Integrated TCP/IP protocol stack
- Integrated TR switch, balun, LNA, power amplifier and matching network
- Integrated PLL, regulators, and power management units
- +19.5dBm output power in 802.11b mode
- Integrated temperature sensor
- Supports antenna diversity
- Power down leakage current of < 10uA
- Integrated low power 32-bit CPU could be used as application processor
- SDIO 2.0, SPI, UART
- STBC, 1×1 MIMO, 2×1 MIMO
- A-MPDU & A-MSDU aggregation & 0.4s guard interval
- Wake up and transmit packets in < 2ms
- Standby power consumption of < 1.0mW (DTIM3)

2.2.2 ESP8266 APPLICATIONS

- Smart power plugs
- Home automation
- Mesh network
- Industrial wireless control
- Baby monitors
- IP Cameras

- Sensor networks
- Wearable electronics
- Wi-Fi location-aware devices
- Security ID tags
- Wi-Fi position system beacons

3.8 Buzzer

Basically, the sound source of a piezoelectric sound component is a piezoelectric diaphragm. A piezoelectric diaphragm consists of a piezoelectric ceramic plate which has electrodes on both sides and a metal plate (brass or stainless steel, etc.). A piezoelectric ceramic plate is attached to a metal plate with adhesives. Applying D.C. voltage between electrodes of a piezoelectric diaphragm causes mechanical distortion due to the piezoelectric effect. For a misshaped piezoelectric element, the distortion of the piezoelectric element expands in a radial direction. And the piezoelectric diaphragm bends toward the direction. The metal plate bonded to the piezoelectric element does not expand. Conversely, when the piezoelectric element shrinks, the piezoelectric diaphragm bends in the direction Thus, when AC voltage is applied across electrodes, the bending is repeated, producing sound waves in the air.

To interface a buzzer the standard transistor interfacing circuit is used. Note that if a different power supply is used for the buzzer, the 0V rails of each power supply must be connected to provide a common reference.

If a battery is used as the power supply, it is worth remembering that piezo sounders draw much less current than buzzers. Buzzers also just have one ‘tone’, whereas a piezo sounder is able to create sounds of many different tones.

To switch on buzzer -high 1

To switch off buzzer -low 1

Notice (Handling) In Using Self Drive Method

1) When the piezoelectric buzzer is set to produce intermittent sounds, sound may be heard continuously even when the self drive circuit is turned ON / OFF at the "X" point shown in Fig. 9. This is because of the failure of turning off the feedback voltage.

2) Build a circuit of the piezoelectric sounder exactly as per the recommended circuit shown in the catalog of the transistor and circuit constants are designed to ensure stable oscillation of the piezoelectric sounder.

3) Design switching which ensures direct power switching.

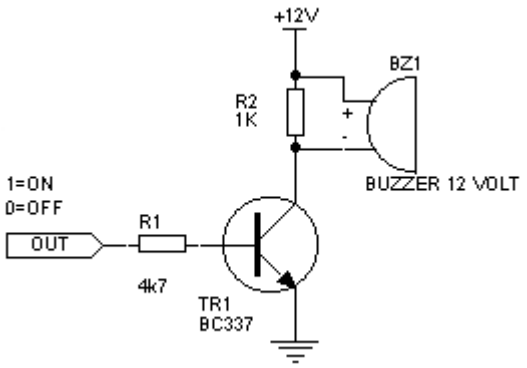
4) The self drive circuit is already contained in the piezoelectric buzzer. So there is no need to prepare another circuit to drive the piezoelectric buzzer.

5) Rated voltage (3.0 to 20Vdc) must be maintained. Products which can operate with voltage higher than 20Vdc are also available.

6) Do not place resistors in series with the power source, as this may cause abnormal oscillation. If a resistor is essential to adjust sound pressure, place a capacitor (about 1μF) in parallel with the piezo buzzer.

7) Do not close the sound emitting hole on the front side of casing.

8) Carefully install the piezo buzzer so that no obstacle is placed within 15mm from the sound release hole on the front side of the casing.



CHAPTER 7

SOFTWARE DESCRIPTION

The Arduino Software (IDE) makes it easy to write code and upload it to the board offline. We recommend it for users with poor or no internet connection. This software can be used with any Arduino board. here are currently two versions of the Arduino IDE, one is the IDE 2.0.0.

7.1 Arduino IDE – Compiler

here are currently two versions of the Arduino IDE, one is the IDE 1.x.x and the other is IDE 2.x. The IDE 2.x is new major release that is faster and even more powerful to the IDE 1.x.x. In addition to a more modern editor and a more responsive interface it includes advanced features to help users with their coding and debugging.

The following steps can guide you with using the offline IDE (you can choose either IDE 1.x.x or IDE 2.x):

1. Download and install the Arduino Software IDE:

- Arduino IDE 1.x.x (Windows, Mac OS, Linux, Portable IDE for Windows and Linux, ChromeOS).
- Arduino IDE 2.x

2. Connect your Arduino board to your device.

3. Open the Arduino Software (IDE).

The Arduino Integrated Development Environment - or Arduino Software (IDE) - connects to the Arduino boards to upload programs and communicate with them. Programs written using Arduino Software (IDE) are called sketches. These sketches are written in the text editor and are saved with the file extension .ino.

Using the offline IDE 1.x.x

The editor contains the four main areas:

1. A Toolbar with buttons for common functions and a series of menus. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

2. The message area, gives feedback while saving and exporting and also displays errors.

3. The text editor for writing your code.

4. The text console displays text output by the Arduino Software (IDE), including complete error messages and other information.

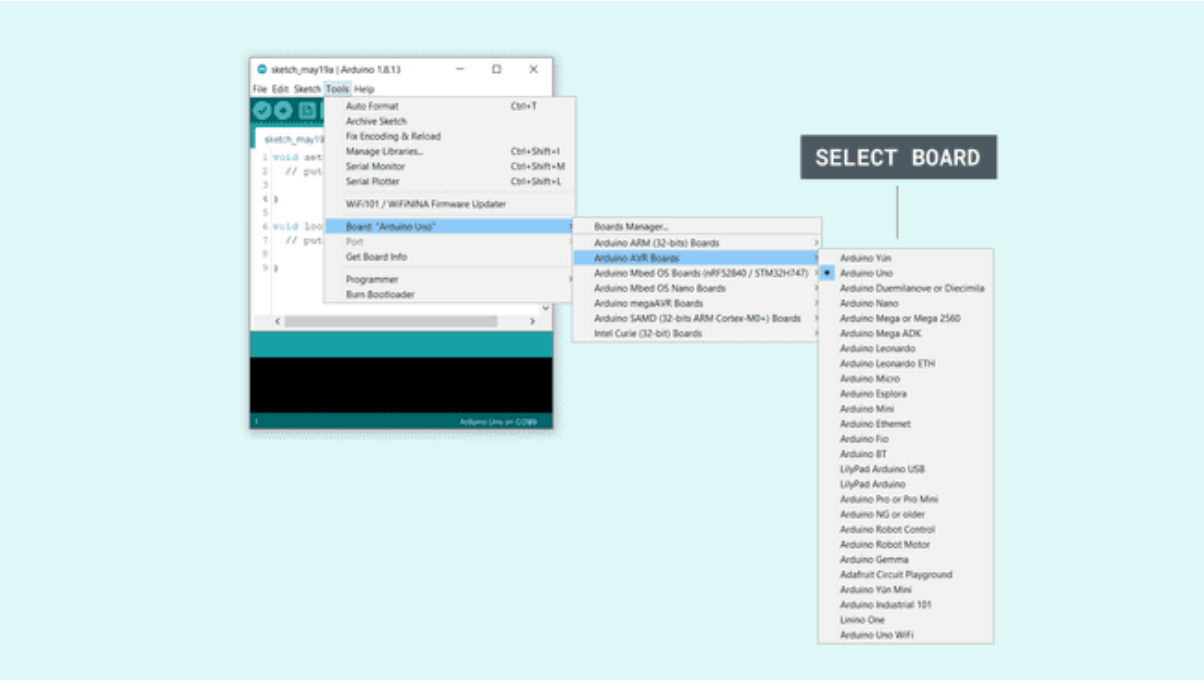
The bottom right-hand corner of the window displays the configured board and serial port.

The Arduino Software IDE

Now that you are all set up, let’s try to make your board blink!

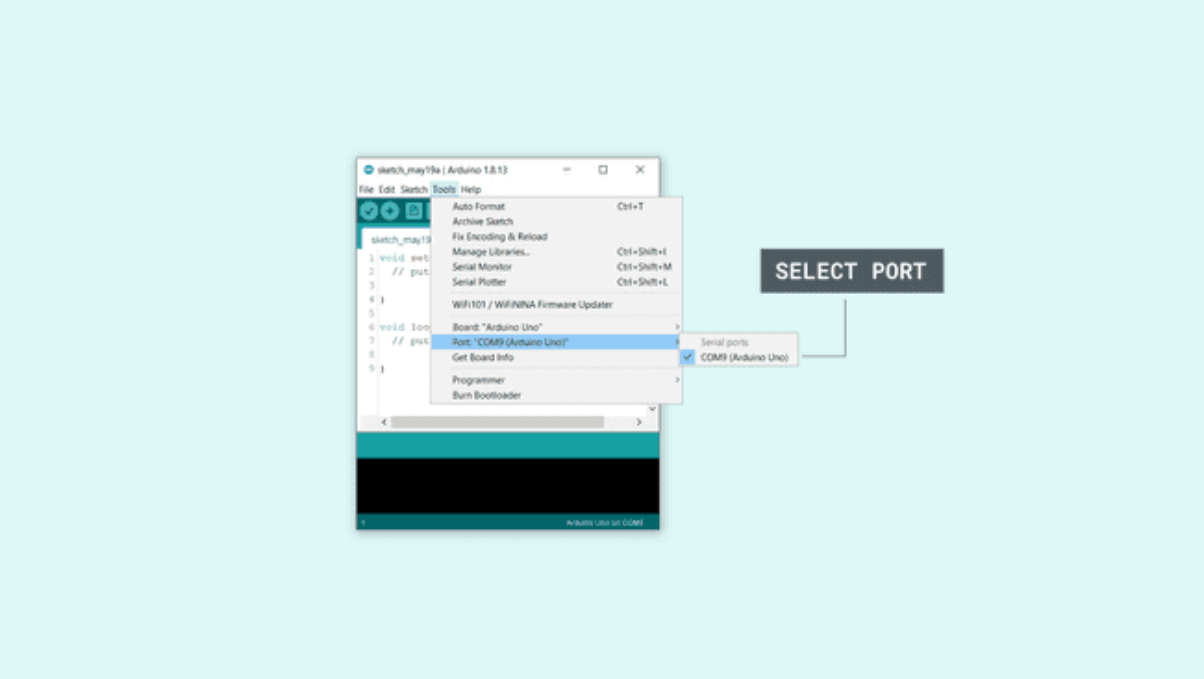
5. Connect your Arduino or Genuino board to your computer.

6. Now, you need to select the right core & board. This is done by navigating to Tools > Board > Arduino AVR Boards > Board. Make sure you select the board that you are using. If you cannot find your board, you can add it from Tools > Board > Boards Manager.



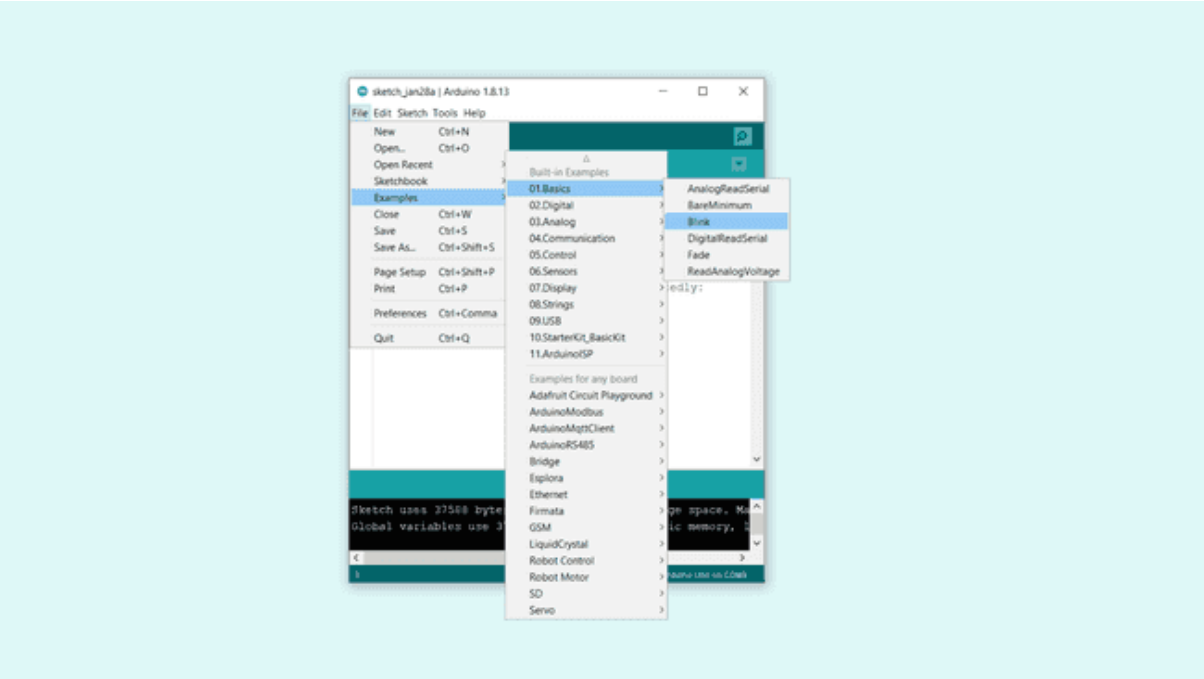
Selecting a board

7. Now, let's make sure that your board is found by the computer, by selecting the port. This is simply done by navigating to Tools > Port, where you select your board from the list.



Selecting the port

8. Let’s try an example: navigate to File > Examples >01.Basics> Blink.



Opening an example

9. To upload it to your board, simply click on the arrow in the top left corner. This process takes a few seconds, and it is important to not disconnect the board during this process. If the upload is successful, the message "Done uploading" will appear in the bottom output area.

10. Once the upload is complete, you should then see on your board the yellow LED with an L next to it start blinking. You can adjust the speed of blinking by changing the delay number in the parenthesis to 100, and upload the Blink sketch again. Now the LED should blink much faster.

The editor contains the four main areas:

1. A toolbar with buttons for common functions and a series of menus. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, choose your board and port and open the serial monitor.

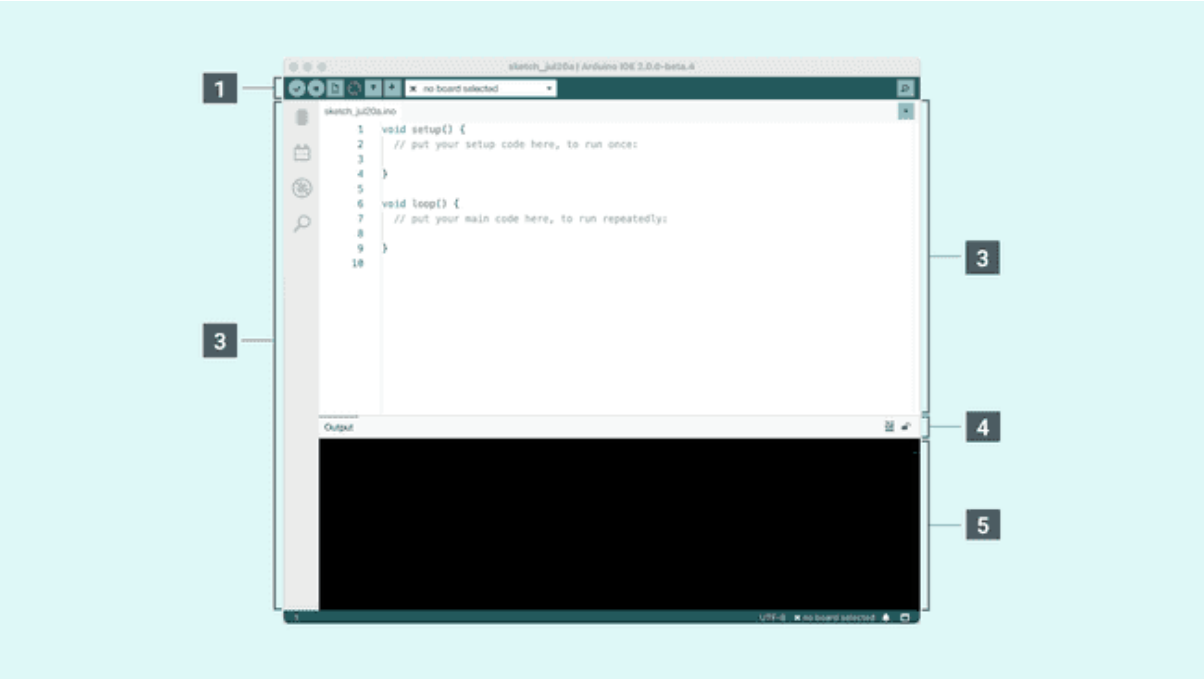
2. The Sidebar for regularly used tools. It gives you quick access to board managers, libraries, debugging your board as well as a search and replacement tool.

3. The text editor for writing your code.

4. Console controls gives control over the output on the console.

5. The text console displays text output by the Arduino Software (IDE), including complete error messages and other information.

The bottom right-hand corner of the window displays the configured board and serial port.



The Arduino Software IDE

Now that you are all set up, let's try to make your board blink!

1. Connect your Arduino or Genuino board to your computer.

2. Now, you need to select the right board & port. This is done from the toolbar. Make sure you select the board that you are using. If you cannot find your board, you can add it from the board manager in the sidebar.

Selecting a board & port

3. To upload it to your board, simply click on the arrow in the top left corner. This process takes a few seconds, and it is important to not disconnect the board during this process. If the upload is successful, the message "Done uploading" will appear in the bottom output area.

CHAPTER 8

SOURCE CODE

```
#include <LiquidCrystal.h>

#include <stdio.h>

LiquidCrystal lcd(13, 12, 14, 27, 26, 25);


#include <WiFi.h>

#include <HTTPClient.h>


HTTPClient http;


const char *ssid = "iotserver";

const char *password = "iotserver123";


int httpResponseCode;

String servername = "http://projectsfactoryserver.in/storedata.php?name=";

String accountname = "iot1372";

String field1 = "&s1=";

String field2 = "&s2=";

String field3 = "&s3=";

String field4 = "&s4=";

String field5 = "&s5=";
```

```
String payload="";


String servername_read = "http://projectsfactoryserver.in/last.php?name=";

String accountname_read = "iot1372L";


int sti=0;

String inputString = "";        // a string to hold incoming data

boolean stringComplete = false;  // whether the string is complete




int units=0,amount=0;

int set_units=10;

int voltage=0;

int current=0;


int sts1=0;

int cntlmk=0;


char gchr,rcv,pastnumber[11];



int mtr     = 22;

int relay1  = 16;

int relay2  = 17;
```

```
int buzzer  = 23;


#define RXD2 0

#define TXD2 3


int swi = 21;

int swe = 19;

int swd = 18;


String theft_string="";


 char gpsval[50];

// char dataread[100] = "";

// char lt[15],ln[15];



 int ii=0,rchkr=0;



void lcdbasic()

{

  lcd.clear();

  lcd.print("U:"); //2,0
```

```
  lcd.setCursor(8,0);

  lcd.print("A:"); //10,0

}


int cntt=0,pd=0;

char pwd[5];


void keypad()

{cntt=0;

   while(1)

       {

      if(digitalRead(swi) == LOW)

        {delay(700);

       while(digitalRead(swi) == LOW);

            cntt++;

          if(cntt >= 9)

            {

           cntt=9;

            }


        //clcd(0xc0+pd);  conv1(cntt);

        lcd.setCursor(pd,1);convertk(cntt);

      }

      if(digitalRead(swd) == LOW)

        {delay(700);
```

```
  while(digitalRead(swd) == LOW);
      cntt--;
    if(cntt <= 0)
      {
     cntt=0;
    }
   //clcd(0xc0+pd);  conv1(cntt);
   lcd.setCursor(pd,1);convertk(cntt);
  }

 if(digitalRead(swe) == LOW)
   {delay(700);
  while(digitalRead(swe) == LOW);

   pwd[pd] = (cntt+48);
   pd++;

   //clcd(0xc0+pd);
    lcd.setCursor(pd,1);

  }
 }
}
```

```
void beep()
{
  digitalWrite(buzzer, LOW);delay(2000); digitalWrite(buzzer, HIGH);delay(200);
}


void iot_send()
{
     lcd.setCursor(15,0);lcd.print("U");
     http.begin(servername + accountname + field1 + String(units) + field2 + String(amount)
+ field3 + String(set_units));
     httpResponseCode = http.GET();
     if(httpResponseCode>0)
       {
        payload="";
        //Serial.print("HTTP Response code: ");
        //Serial.println(httpResponseCode);
        payload = http.getString();
        //Serial.println(payload);
       }
     else
       {
        ;
        //Serial.print("Error code: ");
        //Serial.println(httpResponseCode);
       }
```

```
    delay(3000);
    lcd.setCursor(15,0);lcd.print(" ");
}
void iot_read()
{
    lcd.setCursor(15,0);lcd.write('R');
    http.begin(servername_read + accountname_read);
    httpResponseCode = http.GET();
    if(httpResponseCode>0)
     {
      payload="";
      //Serial.print("HTTP Response code: ");
      //Serial.println(httpResponseCode);
      payload = http.getString();
      //Serial.println(payload);
      //lcd.clear();lcd.print(payload);

      if(payload == "*1#"){digitalWrite(relay1, HIGH);}
      if(payload == "*2#"){digitalWrite(relay1, LOW);}

      if(payload == "*3#"){digitalWrite(relay2, HIGH);}
      if(payload == "*4#"){digitalWrite(relay2, LOW);}

      if(payload == "*5#"){digitalWrite(relay3, HIGH);}
      if(payload == "*6#"){digitalWrite(relay3, LOW);}
```

```
      }
    else
      {
       ;
       //Serial.print("Error code: ");
       //Serial.println(httpResponseCode);
      }
    delay(1000);
    lcd.setCursor(15,0);lcd.write(' ');
}
void setup()
{
  Serial.begin(9600);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(mtr, INPUT);
  pinMode(relay1, OUTPUT);pinMode(relay2, OUTPUT);pinMode(relay3, OUTPUT);
  pinMode(buzzer, OUTPUT);

  pinMode(swi, INPUT_PULLUP);pinMode(swe, INPUT_PULLUP);pinMode(swd,
INPUT_PULLUP);

  digitalWrite(relay1, LOW);digitalWrite(relay2, LOW);digitalWrite(relay3, LOW);
  digitalWrite(buzzer, HIGH);
```

```
  lcd.begin(16, 2);lcd.cursor();

  lcd.print("    IOT Smart ");

  lcd.setCursor(0,1);

  lcd.print("  Energy Meter ");

  delay(2500);


  WiFi.begin(ssid, password);

  Serial.println("Connecting");

  while(WiFi.status() == WL_CONNECTED)

      {

         delay(500);

      }

  //Serial.println(WiFi.localIP());

  delay(3000);


  lcd.clear();

  lcd.print("U:"); //2,0

  lcd.setCursor(8,0);

  lcd.print("A:"); //10,0


  lcd.setCursor(0,1);

  lcd.print("S:"); //2,1

}


void loop()
```

```
{
  if(digitalRead(mtr) == LOW )
    {delay(100);
     while(digitalRead(mtr) == LOW);


      units++;
      amount = (units * 2);


      lcd.setCursor(2,0); convertl(units);
      lcd.setCursor(10,0);convertl(amount);


  lcd.setCursor(0,1);
  lcd.print("S:"); //2,1
  convertl(set_units);


      if(units >= set_units)
        {
         beep();
        }


      iot_send();


    }


  delay(100);
```

```
/*
voltage = analogRead(34);
voltage = (voltage/13);
lcd.setCursor(2,1); convertl(voltage);


current = analogRead(35);
current = (current/204);
lcd.setCursor(10,1);convertl(current);
*/




if(digitalRead(swe) == LOW)
{
  while(digitalRead(swe) == HIGH);
   delay(500);
   lcd.clear();
   lcd.print("Set Units:");
   lcd.setCursor(11,0);keypad();
   set_units = (((pwd[0]-48)*1000) + ((pwd[1]-48)*100) + ((pwd[2]-48)*10) + (pwd[3]-48));
   delay(1500);



lcd.clear();
lcd.print("U:"); //2,0
```

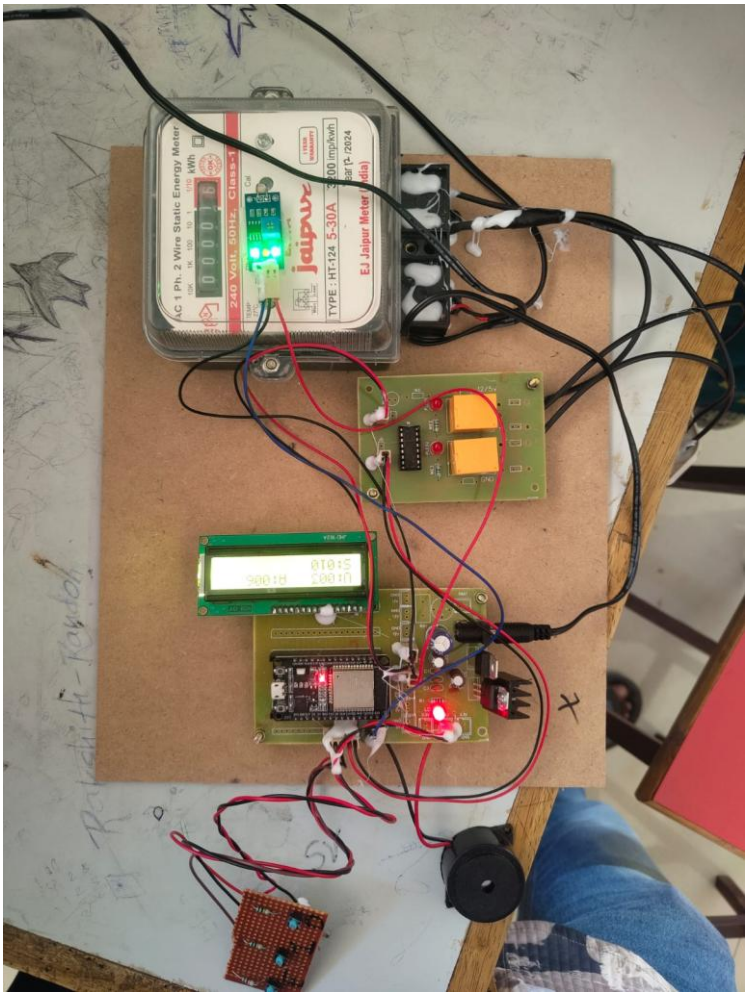
```
  lcd.setCursor(8,0);
  lcd.print("A:"); //10,0

     lcd.setCursor(2,0); convertl(units);
     lcd.setCursor(10,0);convertl(amount);

  lcd.setCursor(0,1);
  lcd.print("S:"); //2,1
  convertl(set_units);

    iot_send();
   }

  cntlmk++;
  if(cntlmk >= 400)
   {cntlmk=0;
     iot_read();
   }
}

void convertl(unsigned int value)
{
  unsigned int a,b,c,d,e,f,g,h;
```

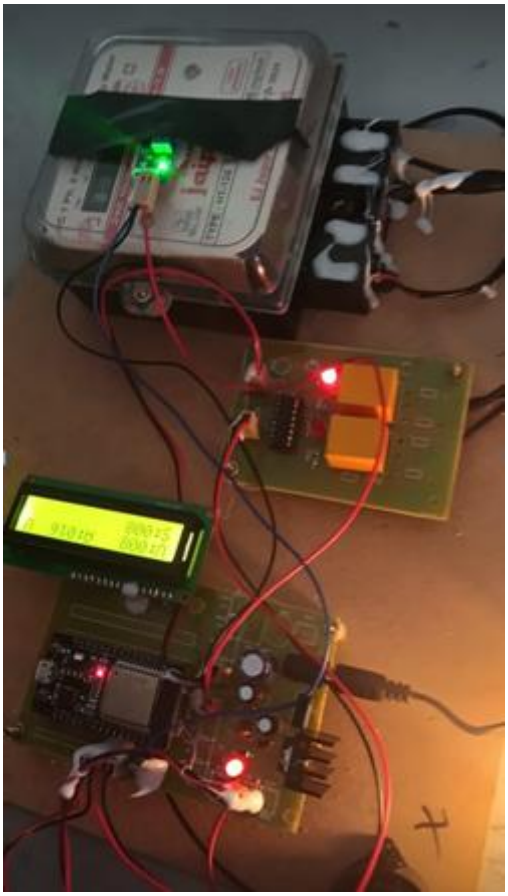
```
    a=value/10000;
    b=value%10000;
    c=b/1000;
    d=b%1000;
    e=d/100;
    f=d%100;
    g=f/10;
    h=f%10;


    a=a|0x30;
    c=c|0x30;
    e=e|0x30;
    g=g|0x30;
    h=h|0x30;


  // lcd.write(a);
  // lcd.write(c);
   lcd.write(e);
   lcd.write(g);
   lcd.write(h);
}
void convertk(unsigned int value)
{
```

```
unsigned int a,b,c,d,e,f,g,h;

   a=value/10000;

   b=value%10000;

   c=b/1000;

   d=b%1000;

   e=d/100;

   f=d%100;

   g=f/10;

   h=f%10;



   a=a|0x30;

   c=c|0x30;

   e=e|0x30;

   g=g|0x30;

   h=h|0x30;



// lcd.write(a);

// lcd.write(c);

// lcd.write(e);

// lcd.write(g);

 lcd.write(h);
```

CHAPTER 9

RESULTS





U:008
S:008
A:016 U

Load1_ON	Load1_OFF	Load2_ON	Load2_OFF

S.No	Units	Amount	Set_units	Date
1	8	16	8	2026-02-10 14:23:41
2	7	14	8	2026-02-10 14:14:07
3	6	12	8	2026-02-10 14:11:27
4	4	8	8	2026-02-10 14:09:30
5	5	10	8	2026-02-10 14:09:30
6	3	6	8	2026-02-10 14:07:10
7	2	4	8	2026-02-10 14:06:04
8	1	2	8	2026-02-10 14:02:54
9	0	0	8	2026-02-10 14:00:52
10	1	2	10	2026-02-10 13:54:04
11	2	4	10	2026-02-10 12:34:48
12	1	2	10	2026-02-10 12:34:35
13	11	22	11	2026-01-30 11:43:18
14	10	20	11	2026-01-30 11:40:28

CHAPTER 10

CONCLUSION

The smart energy meter system efficiently monitors and controls energy consumption using the ESP32 microcontroller. It collects data from an energy meter and allows user input through a keypad. The processed data is displayed on an LCD, while a relay module manages connected loads. A buzzer provides alerts for specific conditions.

The integration of WiFi and cloud connectivity enables remote monitoring and control via a mobile website, offering convenience and real-time access to energy usage information. This system enhances energy efficiency, reduces wastage, and promotes smart energy management for homes and industries**.**

REFERENCES:

[1] **Ravi Kumar** and **Anil Das**, "Identifying Data Security Risks in Smart Metering: Proposing Encryption Techniques," International Journal of Network Security & Its Applications (IJNSA), 2023.

[2] **Akash Chopdar**, **Snehal Yadav Patil**, **Swati Chougule**, and **Rutuja Shewale**, "IoT Based Energy Meter with Current, Voltage and Cost Display," International Journal of Advance Research and Innovative Ideas in Education (IJARIIE), 2022.

[3] **K. Krishna Reddy** and **N.V. Subba Reddy**, "Exploring MQTT Protocol in Smart Metering for Efficient Data Exchange with Cloud Servers," International Journal of Recent Advances in Science and Engineering (IJRASE), 2022.

[4] **Anil Singh**, **Neha Gupta**, and **Amit Kumar**, "Development of a Buzzer-Based Alert System for Excessive Energy Consumption," International Journal of Engineering Research and Technology (IJERT), 2022.

[5] **Imran Khan**, **Sara Ahmed**, and **Mohammed Ali**, "AI-Driven Energy Forecasting for Better Cost Management in Prepaid Meters," International Journal of Artificial Intelligence and Applications (IJAIA), 2021.

[6] **Carlos Gomez**, **Laura Martinez**, and **Miguel Perez**, "A Real-Time Mobile App Interface for User Control Over Energy Consumption," International Journal of Interactive Mobile Technologies (iJIM), 2021.

[7] **Md. Saiful Islam**, **Md. Mahmudul Hasan**, and **Md. Saiful Alam**, "Analyzing Mobile Payment Integration in Smart Meters for Enhanced Consumer Convenience," International Journal of Smart Grid and Clean Energy (IJSGCE), 2020.

[8] **Rakesh Desai**, **Anil Mehta**, and **Priya Sharma**, "Development of a Cloud-Based Energy Meter Dashboard for Historical Data Analysis," International Journal of Cloud Computing and Services Science (IJ-CLOSER), 2020.

[9] **S. Mitra**, **A. Roy**, and **P. Bhattacharya**, "Load Profiling in Smart Meters: Optimizing Peak Demand Management," International Journal of Electrical Power & Energy Systems (IJEPES), 2019.

[10] **Amit Bose**, **Rahul Sharma**, and **Priya Singh**, "Bluetooth-Enabled Meter for Localized Energy Tracking via Mobile Apps," International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering (IJAREEIE), 2018.

[11] **Pratik Patel** and **Ravi Kumar**, "IoT-Based Threshold Alert System for High Energy Consumption," International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET), 2019.

[12] **Nanda Kumar** and **Saha Roy**, "The Role of IoT in Reducing Billing Discrepancies and Improving Utility Revenue," International Journal of Electrical and Electronics Engineering (IJEEE), 2017.